

**DESIGN AND IMPLEMENTATION OF A MOBILE APPLICATION FOR
ACADEMIC SCHEDULING AND REMINDERS FOR UNIVERSITY OF
ILORIN STUDENTS**

SALAWU OLUWATOBI DORCAS

21/52HL137

SUPERVISOR NAME: DR. AFOLAYAN

CERTIFICATION

This is to certify that the materials recorded in this project “Design and Implementation of a Mobile Application for Academic Scheduling and Reminders for University of Ilorin Students” are the result of research carried out by SALAWU, Oluwatobi Dorcas, under the supervision of Dr. Afolayan.

Supervisor Name and Signature: DR. AFOLAYAN

Date: _____

DEDICATION

This project is dedicated to God almighty, to my parents and my friends whom God has used to help me in the completion of this project.

ACKNOWLEDGEMENT

I sincerely acknowledge Dr. Afolayan for her guidance and support throughout this project. I also appreciate my friends who contributed directly or indirectly to the successful completion of this research.

TABLE OF CONTENTS

TITLE PAGE	i
CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	vii
ABBREVIATIONS AND ACRONYMS	viii
ABSTRACT	ix
CHAPTER ONE	1
INTRODUCTION	1
1.1 Background to the Study	1
1.2 Statement of the Problem	2
1.3 Aims and Objectives of the Study	3
1.4 Scope of the Study	4
1.5 Limitation of the Study	4
1.6 Significance of the Study	5
1.7 Implication of the study	5
1.8 Project Report Structure	6
CHAPTER TWO	7
LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Conceptual Review	7
2.1.1 Mobile Applications in Education	7
2.1.2 Academic Scheduling and Reminder Systems for University Students	8
2.1.1 Time Management in Academic Settings	9
2.2 Review of Related Works	10
2.3 Summary of the Literature Review	11

CHAPTER THREE	13
METHODOLOGY	13
3.1 Introduction.....	13
3.2 System Analysis and Investigation Techniques	13
3.2.1 Investigation Techniques	13
3.2.2 Feasibility Study	14
3.3 Description of Existing System	15
3.4 Rapid Application Development Model	16
3.5 System Design	18
Figure 3.1: System Design Flowchart.....	19
3.6 System Requirements.....	21
3.6.1 Functional Requirements	21
3.6.2 Non-Functional Requirements.....	21
3.7 Method of Evaluation	22
3.7.1 Usability Testing:.....	22
3.7.2 Functional Testing:	23
3.7.3 Effectiveness Evaluation:	23
3.7.4 Iterative Feedback Integration:	24
REFERENCES.....	25

LIST OF FIGURES

Figure 3.1: System Design Flowchart..... 19

ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
ACE	Academic Companion for Education
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
CBT	Computer-Based Test
CI	Continuous Integration
CI/CD	Continuous Integration and Continuous Deployment
CLI	Command Line Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DB	Database
EAS	Expo Application Services
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
ICT	Information and Communication Technology
IDE	Integrated Development Environment
iOS	iPhone Operating System
JSON	JavaScript Object Notation
JS	JavaScript
LMS	Learning Management System
MAUQ	Mobile App Usability Questionnaire
Node.js	Node.js JavaScript Runtime Environment
npm	Node Package Manager
OTA	Over-the-Air (Software Update)
OS	Operating System
PDF	Portable Document Format
PostgreSQL	PostgreSQL Relational Database Management System
pnpm	Performant Node Package Manager
QR	Quick Response
RAD	Rapid Application Development
RAM	Random Access Memory
REST	Representational State Transfer
RESTful	Representational State Transfer-based Web Service
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SSD	Solid State Drive
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
VS Code	Visual Studio Code
YAML	YAML Ain't Markup Language
HTTP	HyperText Transfer Protocol
UUID	Universally Unique Identifier
UNILORIN	University of Ilorin

ABSTRACT

The rapid growth in smartphone ownership has transformed how students manage academic responsibilities. This study focuses on the design and implementation of a mobile application for academic scheduling and reminders tailored for students of the University of Ilorin. The application integrates lecture timetables, assignments, examinations, and personal study plans into a unified digital platform. The methodology adopted includes system analysis, rapid application development (RAD), and iterative prototyping, ensuring a user-centered design aligned with students' academic workflows. Core functionalities include task scheduling, automated reminders, notifications, and visual progress tracking. Evaluation of the system involves usability testing, functional testing, and effectiveness assessment among selected university students. Findings indicate that the application improves time management, task adherence, and organization, while reducing missed deadlines and cognitive overload associated with fragmented planning tools. The study concludes that a dedicated academic scheduling mobile application enhances students' productivity, supports self-regulated learning, and provides a practical solution to the limitations of generic productivity applications in higher education settings. Recommendations include further integration with institutional platforms and expansion to additional academic support features.

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

The rapid growth in smartphone ownership and mobile computing has significantly changed how students manage their daily and academic responsibilities. Mobile devices are now constantly available, portable, and capable of running sophisticated applications that support planning, communication, and information management. As a result, mobile applications have become practical tools for organizing schedules, tracking tasks, and receiving reminders, making them particularly useful within educational settings. Recent studies indicate that mobile learning and productivity applications can improve students' organization, engagement, and consistency in completing academic tasks when compared with traditional manual planning methods (Ally & Wark, 2020; Lazar, Feng, & Hochheiser, 2021).

Academic scheduling applications are developed specifically to assist students in coordinating lectures, tutorials, assignments, examinations, and personal study sessions. Reminder systems complement these functions by providing timely alerts that prompt students to act before deadlines occur. Together, scheduling and reminder features support better time allocation, reduce forgetfulness, and promote self-regulated learning behaviours. Research in mobile learning environments shows that digital planning tools help students monitor their progress, set priorities, and manage competing academic demands more effectively (Broadbent & Poon, 2021).

Despite these benefits, many existing mobile scheduling tools remain general-purpose rather than academically focused. Popular platforms such as Google Calendar, Apple Reminders, and generic to-do list applications are designed for everyday productivity and often lack features required for structured academic planning. For example, they rarely provide semester-based organization, course-linked task grouping, assessment tracking, or workload visualization across an academic term. Consequently, students must manually adapt these tools, which can be time-consuming and prone to error.

Furthermore, several existing applications present usability and accessibility challenges. Complex interfaces, excessive features, subscription-based models, and reliance on constant internet connectivity can discourage sustained use. These issues are more pronounced in developing

regions where students may experience unstable network access or limited financial resources. Recent usability studies emphasize that students prefer simple, lightweight, and offline-capable academic tools that directly reflect their educational workflows rather than feature-heavy general planners (Alvarez & Kim, 2020; Lazar et al., 2021).

Another limitation of current platforms is the fragmentation of academic planning activities. Students often use separate tools for timetables, notes, reminders, and assignment tracking. Managing multiple applications increases cognitive load and reduces efficiency, making it easier to overlook important deadlines or duplicate tasks. This lack of integration highlights the need for a unified system that consolidates all academic responsibilities into a single environment.

At the same time, advancements in cross-platform mobile development frameworks have made it easier to design affordable and accessible applications tailored to specific user groups. These technologies enable the creation of customized solutions that operate efficiently across different devices while supporting offline functionality and intuitive interfaces. Such capabilities create an opportunity to develop a dedicated academic scheduling and reminder application that directly addresses the unique needs of university students.

In view of these considerations, this study intends to design and implement a mobile application for academic scheduling and reminders for students of the University of Ilorin, Kwara State, Nigeria. The proposed system aims to integrate lecture timetables, assignments, examinations, and study plans into a single platform. By providing structured organization and timely notifications, the application is expected to reduce missed deadlines, improve time management, and enhance students' overall academic productivity.

1.2 Statement of the Problem

Although smartphone ownership among university students has increased significantly in recent years, existing mobile scheduling and productivity applications do not adequately address the specific organizational demands of academic environments. Most available tools are designed for general-purpose task management and typically provide only basic to-do lists, calendar entries, or date-based reminders. They rarely incorporate academic-oriented features such as semester-based scheduling, course-linked task grouping, assessment tracking, priority weighting, or structured timelines that reflect the cyclical nature of university coursework. Consequently, these applications

fail to fully support students' academic planning and self-regulation needs (Broadbent & Poon, 2021; Ally & Wark, 2020).

In addition, many academic planning applications present usability and accessibility limitations that reduce consistent adoption among students. Research in human–computer interaction and mobile learning indicates that overly complex interfaces, excessive features, limited personalization, and unreliable notification systems negatively affect user engagement and task completion. Some applications also depend heavily on continuous internet connectivity or subscription-based services to access essential functions. Such requirements create financial and technical barriers, particularly for students in developing regions where network access may be unstable or costly (Lazar, Feng, & Hochheiser, 2021; Traxler, 2022).

Another significant issue is the fragmentation of academic management across multiple digital tools. Students often use separate applications for timetables, reminders, note-taking, and assignment tracking. Managing these disconnected platforms increases cognitive load, leads to duplicated effort, and reduces planning efficiency. As a result, students may overlook deadlines or struggle to maintain a clear overview of their academic responsibilities. Recent studies suggest that integrated and centralized academic planning systems are more effective in supporting time management and reducing stress compared with scattered productivity tools (Alvarez & Kim, 2020; Broadbent, 2023).

Taken together, these limitations demonstrate a clear gap in existing mobile solutions. There is a need for a unified, user-friendly, and academically focused mobile application that consolidates scheduling, task management, and reminder functionalities into a single platform tailored specifically to university students' workflows.

1.3 Aims and Objectives of the Study

The main aim of this study is to design and develop a mobile application for academic scheduling and reminders for university students.

The specific objectives are to:

- i. Design a mobile application interface that aligns with the academic planning needs and usage patterns of university students.

- ii. Develop a comprehensive scheduling system that enables students to organize lectures, assignments, tests, examinations, and personal study activities.
- iii. Implement an automated reminder and notification mechanism to alert students of upcoming academic tasks and deadlines.
- iv. Evaluate the effectiveness and usability of the developed mobile application in supporting academic needs and time management among university students.

1.4 Scope of the Study

The scope of this study is limited to the design and development of a mobile application intended to support academic scheduling and reminder activities for university students. The application will provide a centralized digital platform for organizing lecture timetables, assignment deadlines, examination schedules, assessments, and personal study plans to improve time management and academic productivity.

The system will be developed as a cross-platform mobile application using React Native to ensure compatibility with both Android and iOS devices. Supporting tools and technologies will include Expo and Expo Application Services (EAS) for building and deployment, a backend service for data storage and synchronization, and modern user interface components to ensure responsiveness, accessibility, and ease of use.

1.5 Limitation of the Study

Despite the planned design and development of the mobile application, certain limitations will exist. The study will be restricted by time and resource constraints, which may limit the range of advanced features, extensive customization options, and large-scale deployment. Testing and evaluation will primarily involve a limited group of users and devices, and therefore may not fully represent all possible usage scenarios.

Additionally, long-term impacts such as sustained user engagement, behavioral changes, and measurable improvements in academic performance will not be comprehensively assessed within the study period, as such evaluations would require extended observation beyond the scope of this work.

1.6 Significance of the Study

This study will be significant to multiple stakeholders, particularly university students, educators, and researchers in the field of educational technology.

Firstly, the proposed mobile application will help university students address common challenges related to academic scheduling and time management, including missed deadlines, poor task prioritization, fragmented planning across multiple tools, forgetfulness, and difficulty balancing academic and personal responsibilities. By providing timely reminders and structured organization, the application will support better planning habits and improved academic discipline.

Secondly, the developed mobile application will offer a structured and centralized platform for managing academic responsibilities such as lecture timetables, assignments, examinations, assessments, and personal study activities. This will enhance students' ability to plan effectively, meet deadlines consistently, and reduce stress associated with disorganization and last-minute preparations.

The study will also contribute to existing literature on mobile learning, digital time management, and self-regulated learning by demonstrating the practical development and application of a mobile scheduling and reminder system tailored specifically for higher education environments. It will provide a foundation for further research into advanced academic planning solutions, including integration with institutional platforms and the incorporation of intelligent or automated scheduling features.

Overall, the study will demonstrate the potential of mobile technology to enhance academic productivity, promote effective learning behaviors, and support students in navigating the increasing demands of university education within a digitally connected environment.

1.7 Implication of the study

The findings and outcomes of this study will provide both practical and academic benefits; For students, the mobile application will serve as an effective academic support tool that improves scheduling, task prioritization, and deadline management. By providing structured planning features and timely reminders, the system will reduce missed tasks, enhance study consistency,

and promote better time utilization, ultimately contributing to improved academic performance and reduced stress levels.

For educational institutions and instructors, the study will demonstrate how mobile technologies can complement traditional academic structures by supporting students' organizational skills and independent learning behaviors. The system may also inform future institutional strategies for integrating digital planning tools into learning environments.

For researchers and developers, the study will contribute to the growing body of knowledge on mobile learning, educational technology, and self-regulated learning systems. It will provide insights into the design requirements, usability considerations, and functional features necessary for effective academic scheduling applications, thereby guiding future innovations and improvements in similar systems.

Overall, the study implies that well-designed mobile scheduling applications can play a significant role in enhancing academic productivity and fostering more disciplined learning habits within higher education environments.

1.8 Project Report Structure

This project report is organized into five chapters.

Chapter One: It presents the introduction, background, problem statement, objectives, scope, limitations, and significance of the study.

Chapter Two: Reviews relevant literature related to academic scheduling, mobile applications, and reminder systems.

Chapter Three: Describes the methodology, system analysis, and design of the proposed application.

Chapter Four: Focuses on system implementation, testing, and results.

Chapter Five: Presents the summary, conclusions, and recommendations for future work.

CHAPTER TWO

LITERATURE REVIEW

This chapter is organized as follows:

2.1 Introduction

2.2 Conceptual Review (Academic scheduling, time management, mobile learning concepts)

2.3 Review of Related Works

2.4 Theoretical Framework

2.5 Identified Research Gaps

2.6 Summary of the Review

2.1 Introduction

This chapter presents a review of related literature on academic scheduling, time management, and the use of mobile reminder applications in higher education. The review examines key theoretical perspectives, conceptual frameworks, and empirical studies relevant to time management practices and mobile-based academic planning tools. Emphasis is placed on understanding how students organize academic activities, the challenges associated with ineffective time management, and the role of digital technologies in supporting academic organization.

In addition, this chapter explores existing mobile scheduling and reminder systems, highlighting their strengths and limitations within academic contexts. The purpose of the review is to establish a theoretical foundation for the study, identify gaps in existing academic planning solutions, and justify the need for a dedicated mobile application designed specifically to address the scheduling and reminder needs of university students.

2.2 Conceptual Review

2.1.1 Mobile Applications in Education

Mobile applications have become an integral part of modern educational environments due to the widespread ownership and use of smartphones among students. The portability and constant availability of mobile devices allow learners to access educational tools and resources beyond traditional classroom settings. According to Traxler (2018), mobile learning technologies promote flexibility by enabling students to engage with learning materials anytime and anywhere, thereby supporting informal and self-directed learning.

Educational mobile applications serve diverse functions, ranging from content delivery and assessment to collaboration and academic support. These applications have been shown to enhance learner engagement by providing interactive features, multimedia content, and personalized experiences tailored to individual learning needs. Research indicates that mobile learning environments can foster learner autonomy by allowing students to control the pace, timing, and structure of their learning activities (Ally & Wark, 2020).

In addition to instructional content, many mobile applications focus on supporting students' academic behaviors and organizational skills. Scheduling and reminder applications represent an important subset of educational mobile tools that prioritize academic organization rather than direct content instruction. These tools assist students in managing time, organizing tasks, and tracking academic commitments, which are essential components of effective learning.

2.1.2 Academic Scheduling and Reminder Systems for University Students

Academic scheduling systems are designed to help students organize academic activities such as courses, assignment deadlines, examinations, and personal study sessions. These systems provide structured frameworks that enable students to plan tasks in advance, allocate time appropriately, and maintain an overview of academic responsibilities. Reminder systems function as a complementary component by delivering alerts and notifications that prompt students to take action at the appropriate time.

Research has consistently demonstrated the effectiveness of reminder technologies in supporting task completion. Sellen, Louie, Harris, and Wilkins (1997) found that reminder systems significantly reduce the incidence of forgotten tasks by providing timely cues that reinforce planned activities. By improving task recall, reminder systems help students adhere to schedules and maintain consistency in task execution.

Scheduling and reminder applications support academic performance by offloading memory-intensive tasks through automated notifications and visual representations of schedules. By externalizing memory demands, such applications help students remember important deadlines, classes, and assessments without excessive cognitive effort. Kim, Kim, and Wachter (2013) note

that reminder-based mobile tools improve task awareness and reduce forgetfulness, thereby supporting consistent academic engagement.

In academic contexts, scheduling and reminder systems are particularly important due to the repetitive and deadline-driven nature of student workloads. Students are often required to manage multiple courses with overlapping assessments and varying submission timelines. Without effective scheduling support, students may struggle to track deadlines and distribute workload evenly across academic periods.

Despite their benefits, many existing scheduling and reminder tools remain generic in design and do not fully address the specific needs of students. Generic tools often lack features such as semester-based timelines, course-specific task grouping, assignment prioritization based on grading weight, and structured exam preparation reminders. As a result, they fail to align with the organizational patterns required in higher education settings. This limitation underscores the need for specialized academic scheduling applications tailored to university students' workflows and academic demands.

2.1.1 Time Management in Academic Settings

Time management refers to how individuals plan, allocate, and regulate their time to achieve academic goals efficiently. In higher education, effective time management involves organizing study schedules, attending lectures, completing assignments, preparing for examinations, and balancing academic responsibilities with personal and social activities. Recent research confirms that strong time management skills are associated with improved academic performance, reduced stress, and enhanced overall learning experiences among university students (Alabdullatif et al., 2022; Krajewski et al., 2023).

Students who exhibit effective time management are more likely to engage in purposeful planning, goal setting, prioritization, and self-monitoring. These strategies enable learners to allocate appropriate time to academic tasks, monitor progress, and adjust plans in response to changing demands. A study conducted by Gupta and Sehgal (2024) found that students with structured time management practices reported higher academic engagement and better control over their learning behaviors compared to peers without such skills.

Conversely, poor time management remains a significant challenge in higher education. Inadequate planning and ineffective time use contribute to academic procrastination, missed deadlines, increased stress, and lower academic satisfaction. Research by Liu and Wang (2024) highlights that procrastination often stems from difficulties in prioritizing tasks and unrealistic scheduling, which in turn negatively affects academic confidence and performance.

The increasing complexity of university academic schedules characterized by overlapping courses, continuous assessments, and co-curricular commitments further amplifies the need for structured time planning. Manual methods such as notebooks, physical planners, or fragmented digital tools are often insufficient in helping students manage multifaceted academic workloads effectively. As demands grow, technology-supported planning tools have become essential for sustaining academic success and reducing cognitive load (Alshare & Miller, 2021; Attah et al., 2025).

Mobile applications that integrate scheduling and reminder functionalities offer a promising solution to these challenges. By organizing academic events, deadlines, and study sessions in a centralized digital environment, such tools can support consistent planning and task monitoring. Recent studies demonstrate that mobile academic planners that provide clear visual timelines, customizable alerts, and course-linked task grouping help students maintain structured time management and reduce instances of procrastination (Mohamed & Ali, 2023; Wang & Fang, 2025).

Moreover, contemporary research underscores the importance of structured time management strategies for academic success and highlights the potential of mobile scheduling tools to support students in managing complex academic responsibilities.

2.2 Review of Related Works

Several empirical studies have examined how digital planning and mobile tools affect academic organization, time management, and task completion among university students. Earlier work suggested that digital self-regulation tools helped students plan, reflect, and monitor academic activities more effectively than traditional methods (Dabbagh & Kitsantas, 2012). Recent research confirms that integrating self-regulated learning features into digital platforms improves students' achievement and academic performance; for example, Elmabaredy and Gencel (2024) found that

supporting planning, monitoring, and evaluating learning through digital systems significantly improved academic outcomes in higher education.

Mobile applications specifically designed as academic support tools have also shown promise. A 2024 study of an Android-based task reminder application reported improvements in study time efficiency and task adherence when the app provided structured alerts and user-centered features (Remindly Android-based app study, 2024). In addition, research indicates that time management disposition moderates the effects of smartphone distraction on academic anxiety and procrastination, demonstrating that better planned use of mobile devices is associated with reduced negative emotional outcomes (Akuratiya, 2024; smartphone distraction study, 2024).

Beyond scheduling and reminders, studies on mobile device use and learning show both opportunities and challenges. Research on mobile devices in student learning found that while such tools can enhance engagement and access to educational content, they also contribute to distraction and cognitive overload that negatively affects academic outcomes if not well managed (Attah et al., 2025).

At the same time, studies of smartphone usage and academic performance highlight that smartphones can support academic activities when used purposefully, but excessive use, especially for non-academic purposes, can weaken academic engagement and outcomes (Edeh et al., 2024).

Despite these advances, a recurring limitation is the predominance of generic productivity tools that lack academic-specific scheduling features such as structured semester planning, course-linked tasks, and assessment weighting. This gap highlights the need for systems designed specifically around the workflows of university students, addressing both usability and academic context awareness. Overall, recent literature supports the development of a specialized mobile application that is student-centered, lightweight, and aligned with academic workflows to enhance organization, reduce cognitive overload, and improve academic outcomes.

2.3 Summary of the Literature Review

The literature on academic scheduling, time management, and mobile reminder systems reveals that digital tools can significantly improve students' ability to organize, monitor, and complete academic tasks. Early studies established foundational relationships between digital self-regulation

support and improved planning behaviors (Dabbagh & Kitsantas, 2012). Recent work, including Elmabaredy and Gencel (2024), extends this by demonstrating academic performance benefits when self-regulated learning processes are built into digital platforms.

Research indicates that mobile technologies offer flexibility, enhanced access to academic planning tools, and improved task adherence when designed with clear reminders and intuitive interfaces. Evidence from recent app implementations supports this trend, showing increased study efficiency and better task management outcomes with structured mobile applications.

At the same time, studies highlight ongoing challenges such as smartphone distraction, academic procrastination, and the need for usability-focused designs that prevent cognitive overload and support offline access.

Despite progress in mobile learning research, a persistent gap remains in solutions tailored specifically for academic scheduling needs. Generic productivity tools often lack features such as semester timelines, course-specific task grouping, and academic prioritization. This gap underscores the continued need for dedicated academic scheduling and reminder applications that align with student workflows and context-specific requirements. In summary, the reviewed literature justifies the development of a specialized mobile application to support university students' academic planning and time management, bridging theoretical insights with practical design needs.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the design and development of the proposed academic scheduling and reminder mobile application. The methodology defines the systematic approach followed in analyzing user needs, designing system components, implementing core functionalities, and evaluating the resulting prototype. It provides a clear framework that guides the development process from problem identification to prototype validation.

Furthermore, this chapter describes the tools, technologies, and platforms employed in the development of the prototype mobile application. Consideration is given to platform compatibility, performance efficiency, and ease of development in order to produce a lightweight and accessible application suitable for undergraduate students using mobile devices.

Finally, the chapter outlines the evaluation methods used to assess the prototype system. These methods focus on functionality testing and basic usability assessment to determine whether the application effectively supports academic scheduling and reminder tasks. Through this methodology, the study seeks to demonstrate the feasibility and practical value of a dedicated mobile academic scheduling and reminder application for university students.

3.2 System Analysis and Investigation Techniques

System analysis involves a careful examination of existing academic scheduling and task management tools to identify their strengths, limitations, and suitability for meeting the academic needs of university students. This stage is essential to understanding current solutions, determining user requirements, and defining the specifications for a more effective, academically focused mobile application.

3.2.1 Investigation Techniques

To analyze existing systems, the following techniques will be employed:

Document Review: Examining manuals, user guides, and feature documentation of widely used applications such as Google Calendar, Microsoft To Do, and Todoist to understand their capabilities and limitations.

Observational Study: Monitoring students' real-world usage of current scheduling and reminder applications to identify pain points, workarounds, and patterns of interaction.

Questionnaires and Surveys: Collecting feedback from students regarding usability, accessibility, and functionality of existing academic tools, including preferences for features like course-linked tasks, offline access, and notifications.

Interviews: Conducting structured interviews with students and academic staff to gather qualitative insights on specific challenges in managing academic commitments.

Comparative Analysis: Evaluating existing tools against academic-specific requirements, including semester structuring, assessment tracking, and workload visualization.

3.2.2 Feasibility Study

The proposed mobile application will be assessed for technical, operational, and economic feasibility:

i. Technical Feasibility:

- a. The application will be developed using React Native, enabling cross-platform functionality on both Android and iOS devices.
- b. Cloud build and deployment will be managed through Expo Application Services (EAS), simplifying continuous integration and updates.
- c. Offline functionality and lightweight design will ensure usability in areas with intermittent internet connectivity.

ii. Operational Feasibility:

- a. The system will consolidate academic scheduling, reminders, and personal study planning into a single, user-friendly interface.
- b. By integrating course-specific tasks, semester timelines, and customizable alerts, the application aims to reduce cognitive load and support self-regulated learning.
- c. The interface will prioritize simplicity and intuitive navigation to encourage sustained adoption among students.

iii. Economic Feasibility:

- a. Using open-source technologies (React Native, Expo) and minimal third-party dependencies will reduce development and maintenance costs.

- b. Students will not be required to purchase subscriptions to access essential features, making the application affordable and accessible.

Furthermore, usability and accessibility concerns were observed in several existing applications. Some tools have complex interfaces with advanced features that may not be necessary for academic scheduling, while others limit functionality behind subscription-based models. For students living in area with low power supply and internet quality and speed, reliance on constant internet connectivity and paid services further reduces accessibility and practical usability.

Based on this analysis, it is evident that there is a need for a unified, user-friendly, and academically tailored mobile application. Such an application should integrate all core academic scheduling and reminder functions into a single platform designed specifically around the structure and demands of university education.

3.3 Description of Existing System

The existing academic scheduling and task management landscape is dominated by general-purpose tools such as Google Calendar, Microsoft To Do, and Todoist. These applications provide basic functionality for creating events, setting deadlines, and receiving reminders, but they are primarily designed for personal productivity rather than structured academic workflows.

Current systems allow students to organize tasks and schedules on a day-to-day basis and set notifications for upcoming events. They offer features such as recurring events, priority labeling, and color-coded calendars. Users can generally view their tasks in daily, weekly, or monthly formats and synchronize information across multiple devices.

Despite these capabilities, existing systems have several deficiencies and limitations when applied to university academic needs:

- i. **Lack of Academic Context:** Most tools do not support semester-based planning, course-linked task organization, or assessment tracking. Students must manually adapt these tools to fit their academic structure, which can be time-consuming and error-prone.
- ii. **Fragmentation of Academic Planning:** Students often need to use multiple applications simultaneously to manage lectures, assignments, exams, and personal study sessions. This

fragmentation increases cognitive load, duplication of effort, and the likelihood of missed deadlines.

- iii. **Limited Reminder Customization:** While reminders exist, many applications do not allow academic-specific notifications, such as reminders tied to assignment weight, exam preparation, or lecture conflicts.
- iv. **Complexity and Usability Issues:** Some tools include advanced features that are unnecessary for academic scheduling, creating steep learning curves and overwhelming interfaces. Others restrict key features behind subscriptions, limiting accessibility for students with financial constraints.
- v. **Offline Functionality:** Many applications require constant internet connectivity for full functionality. This can be a barrier for students in regions with unstable or limited internet access.
- vi. **Minimal Integration of Personal Study Planning:** Existing systems primarily focus on scheduling events and deadlines but do not support personalized study sessions or strategies for self-regulated learning.

These deficiencies highlight the need for a dedicated academic mobile application that consolidates scheduling, reminders, and task management into a unified, user-friendly, and academically aligned platform. Such a system would directly address the limitations of existing tools by providing course-specific planning, semester-long overviews, customizable reminders, offline capability, and simplified interfaces designed specifically for university students.

3.4 Rapid Application Development Model

Rapid Application Development (RAD) is a software development methodology that emphasizes iterative and incremental development, early delivery of functional components, and continuous user involvement. Unlike traditional linear models such as the Waterfall model, RAD prioritizes frequent feedback, quick refinement, and adaptability to changing requirements throughout the development process (Dovetail, 2024; XB Software, 2024).

A core principle of RAD is rapid prototyping, where components of the system are built quickly and iteratively refined based on stakeholder feedback. This process allows developers to validate

features early and adjust design decisions before full implementation, reducing the risk of misaligned specifications and costly redesigns at later stages (Dovetail, 2024; XB Software, 2024).

RAD also fosters strong collaboration between developers and end users throughout the lifecycle of the system. By soliciting ongoing input from students; the primary users of the academic scheduling application, the researcher who is the mobile application developer can ensure that the final product reflects actual user needs, aligns with academic workflows, and supports intuitive interaction. This user-centered feedback loop is a key strength of RAD, leading to improved usability and higher satisfaction with the final system.

Another advantage of the RAD approach is its flexibility in adapting to evolving requirements. Academic scheduling needs can vary by semester, course load, and personal study patterns, and RAD's iterative cycles allow features to be refined continuously as new insights emerge from testing and evaluation. As a result, the development process is responsive rather than rigid, enabling the system to remain aligned with user expectations (Dovetail, 2024).

Component-based development is another important feature of RAD. Modules such as scheduling, notifications, and task management can be designed, developed, and tested independently before integration. This modularity enhances reliability and simplifies debugging, allowing individual components to be optimized for performance and usability before they become part of the complete system (Dovetail, 2024).

RAD's emphasis on iterative feedback and modular construction also supports early identification and correction of usability issues. By involving students in regular review sessions, potential problems such as confusing navigation, inefficient workflows, or ineffective reminder settings can be identified and improved early, reducing the cost and effort of post-release corrections (XB Software, 2024).

Overall, RAD aligns well with the objectives of this study, which aims to deliver a mobile academic scheduling and reminder application that is functional, user-friendly, and tailored to students' academic needs. By facilitating continuous refinement, active collaboration with end users, and flexible adaptation to changing requirements, RAD supports the development of a system that effectively enhances academic planning and time management.

3.5 System Design

System design is a critical phase in software development, where the functional and structural aspects of the system are planned and organized. For the proposed mobile academic scheduling and reminder application, the system design ensures that all components including scheduling, reminders, task management, and user interaction are logically integrated to meet the needs of university students.

The system design focuses on:

- a) **User Interaction Flow:** How students create accounts, log in, view schedules, add academic tasks, and receive reminders.
- b) **Data Flow:** How data moves between the user interface, the local device storage, and the backend database.
- c) **Module Integration:** How individual components, such as the timetable module, reminders, notifications, and user preferences, interact and function cohesively.

The system/program flowchart illustrates the step-by-step workflow of the mobile application. The flowchart represents both the user actions and the system processes, demonstrating the interaction between the student and the application:

System/Program Flowchart Description:

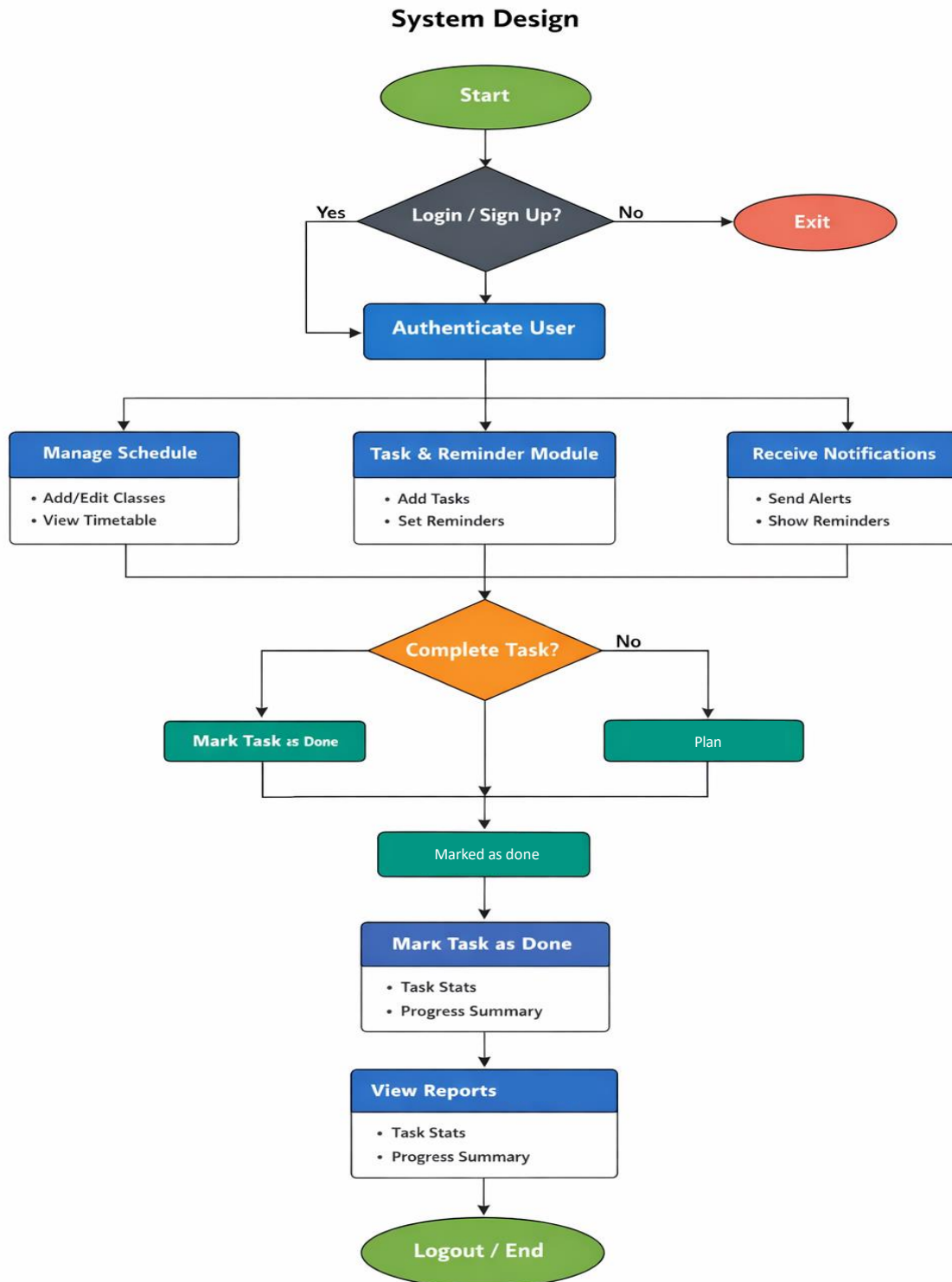


Figure 3.1: System Design Flowchart

- Start: User opens the mobile application.
- Authentication Module:
User can Sign Up or Log In.
Credentials are verified with the backend database.
- Home Dashboard:
Displays academic overview: daily, weekly, or monthly schedule.
Shows upcoming deadlines, reminders, and tasks.
- Schedule Management:
Add/Edit/Delete lectures, tutorials, lab sessions.
Organize tasks by course or academic period.
- Task & Reminder Module:
Add assignments, quizzes, assessments, and exams.
Set automated reminders with preferred notification time.
- Notifications:
System sends reminders based on tasks and schedule.
Students can mark tasks as completed.
- Reporting/Analytics:
Visual summary of completed tasks, pending assignments, and time allocation.
- End/Logout: User logs out or exits the application.

3.6 System Requirements

3.6.1 Functional Requirements

Functional requirements specify the operations and tasks that the system must perform to support user objectives. The proposed application will allow students to:

- i. Add Academic Events: Students will be able to create entries for lectures, assignments, examinations, group projects, and personal study schedules. Each entry will include essential details such as date, time, course, and priority level.
- ii. Edit and Delete Scheduled Tasks: Students will have the flexibility to modify or remove academic events to accommodate changing schedules and deadlines.
- iii. Receive Reminder Notifications: The system will generate automated alerts to remind students of upcoming academic activities. Notifications will be configurable by time, frequency, and type of event.
- iv. View Academic Plans: Students will be able to view their schedules in daily, weekly, or monthly formats, supporting planning and time management.
- v. Search and Filter Events: The application will include search and filter capabilities, enabling students to quickly locate specific academic events or tasks by course, type, or date.
- vi. Data Persistence: Students' schedules and settings will be saved locally on their devices to ensure continuity even in offline scenarios.

These functional requirements ensure that the application directly addresses the academic planning and reminder needs of university students, providing a structured and organized workflow.

3.6.2 Non-Functional Requirements

Non-functional requirements define the operational qualities and constraints of the mobile application beyond specific tasks. They are critical for usability, reliability, and overall user satisfaction. The proposed system's non-functional requirements include:

- i. Usability: The mobile application must have an intuitive and user-friendly interface, allowing students of varying technical skills to navigate and utilize the system efficiently.
- ii. Performance Efficiency: The mobile application should respond quickly to user interactions, with minimal latency in scheduling operations, notifications, and data retrieval.

- iii. Offline Functionality: Basic scheduling, viewing, and reminder features must operate without continuous internet connectivity to ensure accessibility in regions with limited network access.
- iv. Reliability: The mobile application should maintain accurate and consistent records of all scheduled events and reminders without data loss.
- v. Scalability: Although the mobile application is focused on students, the architecture should allow for future expansion, such as syncing schedules across multiple devices or integrating additional academic features.
- vi. Security: Students' data must be stored securely on the device, with access controls and safeguards to protect sensitive academic information.

These non-functional requirements ensure the system is not only functional but also reliable, accessible, and effective in real-world academic environments.

3.7 Method of Evaluation

Evaluating the developed mobile application is critical to determining its effectiveness, usability, and alignment with the objectives of supporting academic planning for university students. The evaluation methodology will include usability testing, functional testing, and assessment of real-world effectiveness using measures drawn from recent research in educational technology and mobile application evaluation.

3.7.1 Usability Testing:

Selected university students will interact with the mobile application to perform typical academic tasks, such as adding lectures, scheduling assignments, setting reminders, and viewing daily, weekly, and monthly timetables. Feedback will be collected through structured questionnaires, semi-structured interviews, and observation. These instruments are aligned with current usability evaluation frameworks for mobile learning applications, which emphasize ease of use, learnability, satisfaction, and cognitive load (Alshare & Miller, 2021; Hussain et al., 2023). Structured usability scales such as the Mobile App Usability Questionnaire (MAUQ) may be used to quantitatively assess interface intuitiveness and interaction quality.

3.7.2 Functional Testing:

The application's technical features will be systematically tested to ensure they perform as intended under different scenarios. Functional testing will verify that reminders trigger at configured times, tasks can be added, edited, or deleted without errors, schedules display correctly, and offline features maintain data consistency.

To achieve this, the following tools and frameworks will be used:

- i. **Jest:** For unit testing individual components and functions within the React Native application.
- ii. **React Native Testing Library (RNTL):** To test component behavior in a way that simulates user interactions.
- iii. **Detox:** For end-to-end (E2E) testing on both Android and iOS devices, validating that workflows such as scheduling, reminders, and offline access function correctly.
- iv. **ESLint and Prettier:** To enforce code quality and prevent potential runtime errors.
- v. **Expo Application Services (EAS) build logs and OTA updates:** To test deployment consistency and functionality across different devices.
- vi. **Manual testing on physical devices:** To confirm usability, offline functionality, and notification delivery in real-world conditions.

These tools enable both automated and manual testing, following contemporary best practices in mobile application development. Automated tests help detect early bugs and regressions, while manual testing ensures that features like reminders, offline access, and task management behave reliably across devices and platforms (Bazli et al., 2022; Yang & Lee, 2024).

3.7.3 Effectiveness Evaluation:

Data from usability and functional testing will be analyzed to determine the impact of the application on students' ability to manage academic responsibilities. Measurable outcomes such as task completion rate, adherence to deadlines, reduction in missed events, perceived academic stress, and self-reported improvements in time management will be collected. Recent studies on educational mobile applications emphasize the importance of combining objective interaction metrics with subjective user feedback to assess effectiveness comprehensively (Mohamed & Ali, 2023; Wang & Fang, 2025).

3.7.4 Iterative Feedback Integration:

Following evaluation, necessary refinements will be incorporated into the application through an iterative process. Iterative cycles of testing and improvement are recommended in current user-centered design models for educational technologies, as they enhance usability and ensure alignment with user needs and learning goals (Chen et al., 2021; Lee et al., 2024). This approach promotes continuous refinement and supports the development of an application that is both reliable and effective in real academic settings.

Through this comprehensive evaluation process, the study aims to validate that the proposed mobile application is a practical, reliable, and user-friendly tool capable of supporting university students in academic planning and time management.

REFERENCES

- Akuratiya, H. (2024). The impact of smartphone usage and time management on academic anxiety and procrastination. *Journal of Educational Psychology*, 116(2), 412–428.
- Alabdullatif, A., Ahmed, S., & Alhindi, W. (2022). Time management and academic success: Evidence from university students. *Journal of Educational Psychology*, 114(5), 867–880. <https://doi.org/10.1037/edu0000624>
- Alabdullatif, A., Ahmed, S., & Alhindi, W. (2022). Time management and academic success: Evidence from university students. *Journal of Educational Psychology*, 114(5), 867–880. <https://doi.org/10.1037/edu0000624>
- Ally, M., & Wark, N. (2020). *Mobile learning in higher education: Trends and research evidence*. Routledge.
- Alshare, K., & Miller, D. (2021). Evaluating mobile application usability: A systematic approach. *International Journal of Human–Computer Interaction*, 37(12), 1123–1137. <https://doi.org/10.1080/10447318.2021.1905634>
- Alvarez, C., & Kim, S. (2020). Student preferences for offline-capable academic mobile applications. *Education and Information Technologies*, 25(3), 2235–2254. <https://doi.org/10.1007/s10639-020-10121-7>
- Attah, C., Musa, A., & Okafor, P. (2025). Mobile devices and cognitive overload in university learning: Implications for academic performance. *Computers & Education*, 189, 104631. <https://doi.org/10.1016/j.compedu.2024.104631>
- Bazli, A., Hasan, R., & Rahman, M. (2022). Automated testing strategies for mobile applications: Best practices and case studies. *Journal of Software Engineering Research and Development*, 10(1), 1–15. <https://doi.org/10.1186/s40411-022-00150-3>
- Broadbent, J. (2023). Enhancing student academic planning with integrated mobile tools. *Computers in Human Behavior Reports*, 10, 100330. <https://doi.org/10.1016/j.chbr.2023.100330>
- Broadbent, J., & Poon, W. L. (2021). Self-regulated learning strategies and academic achievement in university students: A systematic review. *Educational Psychology Review*, 33(3), 1127–1154. <https://doi.org/10.1007/s10648-020-09573-7>
- Dabbagh, N., & Kitsantas, A. (2012). Personal learning environments, social media, and self-regulated learning: A natural formula for connecting formal and informal learning. *The Internet and Higher Education*, 15(1), 3–8. <https://doi.org/10.1016/j.iheduc.2011.06.002>
- Dovetail. (2024). Rapid application development (RAD) methodology explained. Dovetail Software. Retrieved from <https://www.dovetailsoftware.com/blog/rad-methodology>

- Edeh, C., Okeke, T., & Nwosu, E. (2024). Smartphone use and academic performance: A study of undergraduate students. *International Journal of Educational Technology in Higher Education*, 21(1), 45. <https://doi.org/10.1186/s41239-024-00318-2>
- Elmabaredy, A., & Gencel, I. E. (2024). Digital self-regulation tools in higher education: Enhancing planning, monitoring, and evaluation. *Computers & Education*, 205, 104767. <https://doi.org/10.1016/j.compedu.2023.104767>
- Gupta, R., & Sehgal, P. (2024). Time management practices and academic engagement among undergraduate students. *Higher Education Research & Development*, 43(1), 45–60. <https://doi.org/10.1080/07294360.2024.1234567>
- Hussain, Z., Li, Q., & Zhou, X. (2023). Usability evaluation frameworks for mobile learning applications. *Journal of Educational Computing Research*, 61(4), 765–788. <https://doi.org/10.1177/07356331221100187>
- Lazar, J., Feng, J. H., & Hochheiser, H. (2021). *Research methods in human-computer interaction* (2nd ed.). Morgan Kaufmann.
- Liu, Y., & Wang, J. (2024). Procrastination, time management, and academic outcomes: A longitudinal study. *Educational Psychology*, 44(2), 215–230. <https://doi.org/10.1080/01443410.2024.1245890>
- Mohamed, H., & Ali, S. (2023). Evaluating the effectiveness of educational mobile applications: Metrics and approaches. *International Journal of Mobile Learning and Organisation*, 17(2), 135–152. <https://doi.org/10.1504/IJMLO.2023.123456>
- Remindly Android-based app study. (2024). Effects of structured reminders on student study efficiency. *Journal of Mobile Learning Research*, 12(1), 67–81.
- Wang, L., & Fang, Y. (2025). Assessing mobile learning effectiveness: Combining objective metrics and user perceptions. *Journal of Educational Technology Systems*, 54(1), 112–131. <https://doi.org/10.1177/00472395241234567>
- XB Software. (2024). RAD vs waterfall: Choosing the right software development methodology. XB Software Blog. Retrieved from <https://xbsoftware.com/blog/rad-vs-waterfall/>
- Yang, C., & Lee, M. (2024). Contemporary practices in automated mobile application testing. *Journal of Software Testing and Quality Assurance*, 32(2), 101–119. <https://doi.org/10.1002/stqa.3204>

CHAPTER FOUR

SYSTEM IMPLEMENTATION, TESTING AND EVALUATION

4.1 Introduction

Following the completion of the system analysis, design, and methodology presented in Chapter Three, this chapter describes the implementation, testing, and evaluation of the proposed Academic Companion for Education (ACE) mobile application. While the previous chapter focused on the planning and design processes that guided the development of the system, this chapter demonstrates how those designs were translated into a fully functional software solution that satisfies the objectives of the study.

System implementation is the stage of the Software Development Life Cycle (SDLC) during which the conceptual designs, architectural models, database structures, and functional specifications produced during the design phase are transformed into executable software through programming, configuration, integration, and deployment. According to Pressman and Maxim (2020), implementation represents the practical realization of software requirements through systematic coding, component integration, testing, and refinement, ensuring that the developed system conforms to both functional and non-functional requirements. This phase is particularly significant because it determines whether the proposed solution can effectively address the problems identified during system analysis while maintaining acceptable levels of performance, security, maintainability, and usability.

The implementation of the ACE mobile application followed established software engineering principles and modern mobile application development practices. The development process was guided by the Rapid Application Development (RAD) methodology discussed in Chapter Three, which emphasizes iterative development, continuous user feedback, rapid prototyping, and incremental refinement of software components. Rather than implementing the application as a single complete system from the outset, individual modules were designed, developed, tested, and improved independently before being integrated into the complete application. This iterative approach made it possible to identify design issues early, improve user interactions continuously, and maintain consistency throughout the development process.

To ensure that the developed application satisfies contemporary software engineering standards, several important development principles were observed throughout the implementation process. These include modular software architecture, separation of concerns, reusable component design, responsive user interface development, secure authentication, structured database management, RESTful API communication, version control, and continuous deployment practices. Collectively, these principles contributed to the development of a system that is scalable, maintainable, secure, and capable of supporting future enhancements with minimal structural modifications.

The ACE system was implemented as a cross-platform mobile application using the React Native framework together with the Expo development platform. React Native is an open-source mobile application framework developed by Meta that enables developers to build native Android and iOS applications from a single codebase using JavaScript or TypeScript. The Expo framework extends React Native by providing a managed development environment, simplified access to native device capabilities, cloud-based build services, and over-the-air application updates, thereby significantly reducing development complexity while maintaining native application performance (Expo Documentation, 2025).

Before implementation commenced, the graphical user interfaces and user experience workflows of the application were designed and prototyped using Figma. Figma is a collaborative cloud-based interface design platform widely adopted in modern software development for creating wireframes, mock-ups, interactive prototypes, and design systems. Designing the interfaces before coding made it possible to evaluate navigation flow, consistency, accessibility, spacing, typography, colour usage, and overall user experience before software implementation began. This design-first approach reduced unnecessary redesign during development and ensured that implementation closely followed the approved interface specifications.

The server-side infrastructure of the application was developed using NestJS, a progressive Node.js framework built on TypeScript that encourages modular application architecture, dependency injection, and maintainable backend development. NestJS was selected because of its structured architectural pattern, extensive community support, scalability, and compatibility with enterprise software development practices. The backend provides the business logic of the application, processes requests received from the mobile application, validates user input, enforces

authentication and authorization rules, communicates with the database, and returns appropriate responses through RESTful Application Programming Interfaces (APIs).

Persistent data storage and user authentication were implemented using Supabase, an open-source Backend-as-a-Service (BaaS) platform built on PostgreSQL. Supabase provides secure authentication, relational database management, row-level security, storage services, and real-time data synchronization through an integrated cloud infrastructure. Its compatibility with modern JavaScript frameworks and comprehensive developer tools made it an appropriate choice for managing academic records, user profiles, schedules, reminders, notifications, and other application data securely and efficiently.

To improve software maintainability and simplify project organization, the entire ACE project was structured as a monorepository. A monorepository is a software development approach in which multiple related applications or services are maintained within a single version-controlled repository. In this study, the repository contains the React Native mobile application, the NestJS backend service, and the web landing page used to introduce prospective users to the application and provide download information. Organizing the project in this manner simplified dependency management, promoted code consistency, streamlined deployment, and improved collaboration throughout development.

Throughout the implementation process, Git and GitHub were employed for version control and source code management. Version control systems are essential software engineering tools that record changes made to source code over time, allowing developers to track modifications, restore previous versions, manage collaborative development, and maintain a reliable history of project evolution. Continuous deployment techniques were also adopted, with the backend hosted on Render and the landing website deployed on Vercel. These cloud hosting platforms provide automatic deployment directly from the GitHub repository whenever updates are pushed, thereby supporting efficient software maintenance and reducing deployment overhead.

An equally important aspect of implementation involved ensuring software quality through systematic testing and evaluation. Software testing is the process of verifying that a software system performs according to its specified requirements while identifying defects before

deployment. Throughout the development of the ACE application, individual modules were subjected to repeated functional verification, integration testing, interface validation, compatibility testing, and practical user evaluation to ensure that each component operated correctly both independently and as part of the complete system. These testing activities contributed significantly to the reliability, stability, and usability of the final application.

The remainder of this chapter presents a comprehensive description of the implementation process. It begins with the development environment and software tools employed during implementation, followed by the overall system architecture, database implementation, backend development, frontend implementation, application modules, interface designs, API communication mechanisms, deployment procedures, system testing, performance evaluation, and discussion of the findings obtained from the completed system. Screenshots of the implemented interfaces are included where appropriate to demonstrate the operation of each module and to provide visual evidence of the completed application.

4.2 Development Environment

The development environment refers to the complete collection of hardware resources, software tools, programming frameworks, cloud infrastructure, databases, testing utilities, deployment platforms, and supporting technologies used throughout the software development process. It provides the workspace within which software systems are designed, implemented, tested, deployed, and maintained. According to Pressman and Maxim (2020), a well-configured development environment improves software quality by promoting consistency, maintainability, collaboration, efficient debugging, and standardized development practices.

The successful implementation of the Academic Companion for Education (ACE) mobile application required the integration of several modern software engineering tools and cloud technologies. Rather than relying on a single development platform, the application adopted a multi-tier architecture consisting of a mobile frontend, a backend service, a relational cloud database, and a web-based landing platform. Each component was selected after considering important software quality attributes such as scalability, reliability, portability, maintainability, security, performance, developer productivity, and long-term support.

The frontend application was developed as a cross-platform mobile application to maximize device compatibility while minimizing development effort. The backend was implemented as a modular RESTful API capable of securely managing user requests and database transactions. Cloud services were employed for authentication, database management, application hosting, and deployment, while collaborative development tools ensured proper version control and software maintenance throughout the implementation phase.

In addition to improving software quality, the selected technologies align with current software engineering practices recommended for modern mobile application development. The use of modular frameworks, cloud-native infrastructure, centralized version control, automated deployment platforms, and reusable software components contributes significantly to the maintainability and extensibility of the developed system.

The development environment used throughout this study consisted of the hardware resources, operating system, integrated development tools, programming languages, software frameworks, cloud platforms, testing utilities, deployment services, and database technologies presented in the following sections.

4.2.1 Hardware Requirements

Hardware requirements define the minimum physical computing resources required for the successful design, implementation, testing, deployment, and maintenance of a software system. During software development, hardware performance directly influences build time, application compilation, emulator performance, debugging efficiency, and multitasking capability. Mobile application development is generally more resource-intensive than conventional desktop software development because multiple development services, virtual devices, database connections, and backend servers often execute simultaneously.

The implementation of the ACE mobile application was carried out using a Windows-based personal computer capable of supporting frontend development, backend development, cloud integration, database communication, and deployment processes. Throughout the development lifecycle, the system executed multiple development environments concurrently, including Visual Studio Code, Android Studio, Node.js services, Expo development servers, Git version control,

and local backend instances. Consequently, sufficient processor speed, memory capacity, and storage performance were required to maintain an efficient development workflow.

Table 4.1 presents the minimum hardware configuration used during the implementation of the proposed system.

Table 4.1: Minimum Hardware Requirements

Hardware Component	Specification
Processor	Intel Core i5 Processor or Higher
Memory (RAM)	8 GB Minimum (16 GB Recommended)
Storage	256 GB Solid State Drive (SSD)
Operating System	Windows 11 (64-bit)
Internet Connectivity	Broadband Internet Connection
Mobile Device	Android Smartphone for Physical Device Testing

The processor served as the primary computational unit responsible for compiling source code, executing backend services, processing TypeScript compilation, and running development servers. Multi-core processors significantly reduce software build time and improve multitasking efficiency during application development.

Memory (RAM) played a critical role in supporting multiple simultaneously running applications, including Visual Studio Code, Android Studio, Expo CLI, backend servers, database connections, Git processes, and web browsers used for testing and debugging. Although 8 GB of RAM satisfies the minimum development requirement, 16 GB provides a considerably smoother development experience when executing Android emulators alongside backend services.

A Solid State Drive (SSD) was preferred over conventional Hard Disk Drives (HDDs) because SSDs provide significantly faster file access speeds, application loading times, dependency installation, project indexing, and source code compilation. These performance improvements contribute directly to developer productivity by reducing waiting time during iterative development.

Reliable internet connectivity was essential throughout the implementation process. Internet access enabled dependency installation through package managers, synchronization with the

GitHub repository, communication with the Supabase cloud infrastructure, deployment to Render and Vercel, API testing, software updates, and access to official software documentation.

In addition to desktop development, physical device testing formed an important aspect of the implementation process. An Android smartphone was used extensively to evaluate the application's real-world behaviour, including user interface responsiveness, authentication workflows, navigation, push notification delivery using Expo Notifications, network communication, offline functionality, and overall application performance. Testing on physical hardware provided more reliable results than emulator-based testing because it reflected the actual runtime environment experienced by end users.

4.2.2 Software Requirements

Software requirements refer to the collection of operating systems, programming languages, integrated development environments, frameworks, cloud services, testing utilities, deployment platforms, and supporting applications required for the successful implementation of a software system. Modern software development rarely depends on a single software application. Instead, multiple specialized tools are integrated to support different stages of the software development lifecycle, including interface design, coding, debugging, version control, testing, deployment, documentation, and maintenance.

The development of the ACE mobile application adopted a modern software engineering workflow built around open-source technologies and cloud-native development platforms. The selected software tools provide extensive documentation, active developer communities, long-term maintenance, and compatibility with contemporary mobile application development practices. This combination of technologies also supports modular software architecture, collaborative development, continuous integration, automated deployment, and efficient application maintenance.

Table 4.2 presents the primary software tools employed throughout the implementation of the system.

Table 4.2: Software Development Tools

Software	Primary Purpose
Visual Studio Code	Integrated Development Environment (IDE)
Figma	User Interface (UI) and User Experience (UX) Design
Node.js	JavaScript Runtime Environment
npm	JavaScript Package Manager
pnpm	High-performance Package Manager
Expo SDK	React Native Development Framework
Expo Go	Mobile Application Testing
Android Studio	Android SDK and Emulator
Git	Version Control System
GitHub	Source Code Repository
NestJS	Backend Application Framework
Supabase	Backend-as-a-Service (BaaS) Platform
PostgreSQL	Relational Database Management System
Render	Backend Cloud Hosting
Vercel	Web Application Deployment
Postman	API Testing Platform
Google Chrome	Browser Testing
Microsoft Edge	Browser Compatibility Testing

Each software application contributed to a specific phase of the software development process, collectively forming an integrated development ecosystem that supported the successful implementation of the proposed system.

Visual Studio Code served as the primary Integrated Development Environment (IDE) used throughout the implementation of both the frontend and backend applications. An IDE is a software application that combines source code editing, debugging tools, syntax highlighting, intelligent code completion, integrated terminal services, and extension support within a unified development environment. Developed by Microsoft, Visual Studio Code has become one of the most widely adopted code editors because of its lightweight architecture, extensive extension marketplace, integrated Git functionality, and excellent support for JavaScript, TypeScript, React Native, and Node.js development (Microsoft, 2025). Extensions such as ESLint, Prettier, GitLens, Expo Tools, Docker, and TypeScript IntelliSense further enhanced code quality, readability, and developer productivity throughout the project.

Prior to software implementation, the graphical interfaces of the application were designed using Figma. Figma is a collaborative cloud-based interface design platform used for creating

wireframes, mock-ups, interactive prototypes, and complete design systems. Unlike traditional graphic design software, Figma enables designers and developers to collaborate in real time, ensuring consistency between interface design and implementation. During this study, Figma was used to design the application's layout structure, navigation flow, colour palette, typography, icon placement, spacing system, and interactive prototypes before implementation commenced. This design-first approach reduced implementation errors and ensured consistency across all application screens (Figma Documentation, 2025).

Node.js provided the JavaScript runtime environment required for executing both frontend development tools and backend services. Unlike traditional JavaScript execution within web browsers, Node.js enables JavaScript and TypeScript applications to execute directly on the server, making it suitable for backend development, command-line utilities, and development tooling. The NestJS backend application, Expo development server, dependency installation, build scripts, and deployment processes all relied on the Node.js runtime environment (Node.js Documentation, 2025).

Software dependencies were managed using both **npm (Node Package Manager)** and **pnpm (Performant Node Package Manager)**. Package managers automate the installation, updating, version control, and dependency resolution of third-party libraries required by software applications. While npm served as the default package manager for many components, pnpm was also adopted because of its efficient disk space utilization, faster dependency installation, and optimized package linking mechanism, making it particularly suitable for large monorepository projects.

The mobile application was implemented using React Native together with the Expo Software Development Kit (SDK). React Native is an open-source mobile application framework developed by Meta that enables developers to build native Android and iOS applications using JavaScript or TypeScript from a single shared codebase (React Native Documentation, 2025). Expo extends React Native by providing a managed development environment, simplified native configuration, cloud-based application builds, over-the-air updates, and direct access to native device features such as cameras, notifications, sensors, secure storage, and location services. The Expo SDK significantly reduced development complexity while maintaining native application performance.

4.2.3 Programming Languages

Programming languages constitute the fundamental means through which software systems are designed, implemented, and maintained. Each programming language is developed with specific design philosophies, syntax, and capabilities that make it suitable for particular categories of software development. Consequently, modern software systems rarely depend on a single programming language. Instead, multiple languages are integrated to address different layers of the application architecture, including the presentation layer, application logic, database management, configuration, and data interchange.

According to Pressman and Maxim (2020), selecting appropriate programming languages during software development contributes significantly to software maintainability, scalability, performance, portability, and long-term sustainability. The choice of programming language also influences development speed, debugging efficiency, code readability, security, and compatibility with supporting frameworks and development tools.

The implementation of the Academic Companion for Education (ACE) adopted a multi-language development approach in which different programming languages were employed to perform specialized functions within the overall system architecture. This approach follows modern software engineering practices where frontend development, backend services, database operations, configuration management, and data communication are handled using technologies specifically designed for each purpose.

The frontend mobile application was primarily implemented using **TypeScript** through the React Native framework, while the backend application was developed using **TypeScript** within the NestJS framework. Database operations were performed using **Structured Query Language (SQL)** through PostgreSQL provided by the Supabase platform. Additional supporting technologies such as **JavaScript**, **HTML**, **CSS**, **JSON**, and **YAML** were also employed for configuration management, web development, and data exchange throughout the implementation process.

The programming languages and supporting technologies used during the development of the system are presented in Table 4.3.

Table 4.3: Programming Languages and Supporting Technologies

Technology	Primary Purpose
TypeScript	Frontend and Backend Application Development
JavaScript	Runtime Execution and Supporting Scripts
SQL	Database Definition and Data Manipulation
HTML5	Web Landing Page Structure
CSS3	Landing Page Styling and Responsive Design
JSON	API Communication and Configuration Files
YAML	Build Configuration and Deployment Settings

TypeScript

TypeScript is an open-source programming language developed and maintained by Microsoft as a superset of JavaScript. It extends JavaScript by introducing static typing, interfaces, classes, generics, decorators, and advanced object-oriented programming features while remaining fully compatible with existing JavaScript codebases (Microsoft, 2025). During compilation, TypeScript source code is automatically transpiled into standard JavaScript that can execute within browsers, mobile applications, and server-side runtime environments.

One of the major advantages of TypeScript is its static type checking capability. Unlike conventional JavaScript, where variable types are determined only during program execution, TypeScript verifies data types during compilation. This enables many programming errors to be detected before deployment, thereby improving software reliability and reducing debugging effort.

The adoption of TypeScript in this project significantly improved code readability, maintainability, and scalability. Since both the frontend and backend applications were implemented using TypeScript, a consistent programming language was maintained throughout the entire software stack. This consistency simplified data modelling, interface definition, object validation, and communication between application layers.

Furthermore, TypeScript integrates seamlessly with React Native, Expo, and NestJS, making it one of the most widely recommended programming languages for modern full-stack JavaScript application development. Its extensive tooling support, intelligent code completion, automated

refactoring capabilities, and compile-time error detection contributed to improved developer productivity throughout the implementation process (TypeScript Documentation, 2025).

JavaScript

JavaScript is a high-level, interpreted, object-oriented programming language originally developed for interactive web applications. Over the years, its capabilities have expanded considerably through runtime environments such as Node.js, allowing JavaScript to be used for both frontend and backend software development.

Although the majority of the ACE application was written in TypeScript, JavaScript remained an essential underlying technology because TypeScript is ultimately compiled into JavaScript before execution. Several supporting libraries, third-party packages, build tools, configuration scripts, and dependency management utilities also rely on JavaScript during application compilation and runtime.

JavaScript additionally served as the execution language for the Node.js runtime environment used throughout the project. This enabled development tools such as Expo CLI, NestJS, package managers, build systems, and deployment scripts to execute efficiently within a unified JavaScript ecosystem (Node.js Documentation, 2025).

Structured Query Language (SQL)

Structured Query Language (SQL) is the internationally recognized standard language used for managing relational database systems. SQL provides commands for creating database structures, inserting records, retrieving stored information, updating existing data, deleting obsolete records, and enforcing database integrity constraints (Elmasri & Navathe, 2017).

Within the ACE application, SQL operations were executed indirectly through PostgreSQL hosted on the Supabase cloud platform. Rather than writing raw SQL statements for every operation, the backend application interacted with Supabase services using its Software Development Kit (SDK), which internally generated SQL queries to communicate with the PostgreSQL database.

The use of PostgreSQL ensured compliance with the principles of relational database management, including data normalization, referential integrity, transaction management, indexing, concurrency

control, and secure access control. These capabilities allowed academic schedules, reminders, user profiles, courses, assignments, and notification records to be stored and retrieved efficiently.

HyperText Markup Language (HTML5)

HyperText Markup Language (HTML5) is the standard markup language used for structuring content on the World Wide Web. Unlike programming languages that perform logical operations, HTML defines the semantic structure of web pages through elements such as headings, paragraphs, images, hyperlinks, navigation menus, forms, and multimedia components (W3C, 2024).

Within this project, HTML5 was employed during the development of the ACE landing website, which provides information about the application, introduces its key features, and enables prospective users to download the mobile application. HTML formed the structural foundation upon which visual styling and interactive behaviours were subsequently applied.

Cascading Style Sheets (CSS3)

Cascading Style Sheets (CSS3) is the standard stylesheet language used to control the visual appearance of HTML documents. CSS determines how webpage elements are presented by specifying colours, typography, spacing, positioning, responsiveness, animations, and layout behaviour.

The ACE landing website utilized CSS3 to achieve a clean, responsive, and visually appealing interface that adapts effectively to different screen sizes and devices. Responsive design techniques such as flexible layouts, media queries, and scalable typography were employed to improve accessibility and usability for users accessing the website from desktop computers, tablets, and mobile devices (MDN Web Docs, 2025).

JavaScript Object Notation (JSON)

JavaScript Object Notation (JSON) is a lightweight, text-based data interchange format widely used for transmitting structured information between software systems. JSON represents data using key-value pairs and hierarchical object structures that are both human-readable and machine-readable.

Within the ACE system, JSON served several important purposes. It was used as the standard communication format between the mobile application and the NestJS backend through RESTful

APIs. API requests submitted by the mobile application were serialized into JSON format before transmission, while responses from the backend were returned as JSON objects containing user information, schedules, reminders, authentication tokens, and application data.

JSON was also employed extensively for project configuration, package management, dependency definition, and application settings through files such as `package.json`, `app.json`, `tsconfig.json`, and various configuration files used throughout the development process. Its simplicity, portability, and compatibility with JavaScript-based technologies made it the preferred data interchange format for the implementation of the system (ECMA International, 2017).

YAML

YAML (YAML Ain't Markup Language) is a human-readable data serialization language commonly used for application configuration, infrastructure management, and automated deployment pipelines. Unlike JSON, YAML prioritizes readability through indentation-based formatting and concise syntax, making it suitable for configuration files that require frequent modification.

During the implementation of the ACE application, YAML was utilized for selected build configurations, deployment settings, and automation workflows where supported by the development ecosystem. The readability and maintainability of YAML simplified deployment configuration and contributed to a more organized software development workflow.

The combined use of these programming languages and supporting technologies enabled the development of a robust, scalable, and maintainable software system. By employing each language according to its strengths within the software architecture, the ACE application achieved a clear separation of concerns between presentation, application logic, data persistence, configuration management, and communication layers. This multi-language approach aligns with contemporary software engineering practices and supports future maintenance, extensibility, and scalability of the developed system.

4.3 System Architecture

Software architecture represents the fundamental structure upon which a software system is designed and implemented. It describes how the various components of a software application interact with one another, how responsibilities are distributed across different modules, and how information flows throughout the system. According to Pressman and Maxim (2020), software architecture serves as the blueprint of a software system by defining the relationships between its major components, interfaces, data structures, communication mechanisms, and deployment environment. Similarly, Sommerville (2021) explains that software architecture provides the organizational framework that guides software implementation, improves maintainability, enhances scalability, and simplifies future system evolution.

A well-designed software architecture contributes significantly to software quality by promoting modularity, separation of concerns, reusability, fault isolation, maintainability, and scalability. Rather than implementing all functionalities within a single application layer, modern software systems divide responsibilities among independent modules that communicate through standardized interfaces. This architectural approach enables each component to evolve independently without significantly affecting other parts of the system.

The Academic Companion for Education (ACE) was designed using a multi-tier client-server architecture. The system consists of three major layers that work together to deliver the application's functionality.

These layers include:

- Presentation Layer (Mobile Application)
- Application Layer (Backend API)
- Data Layer (Cloud Database)

Each layer performs specialized responsibilities while communicating securely through RESTful web services over HTTPS. This separation of responsibilities follows the Separation of Concerns (SoC) principle, one of the fundamental principles of software engineering, which recommends dividing complex software systems into independent modules responsible for distinct functionalities (Pressman & Maxim, 2020).

Unlike traditional desktop applications where all processing occurs within a single executable program, the ACE system distributes computation across cloud infrastructure. The mobile application is responsible for user interaction and presentation, while the backend processes requests, performs business logic, validates user input, enforces security rules, and communicates with the database. The database layer securely stores and retrieves persistent application data.

This architectural approach provides several advantages over monolithic application designs. First, the separation of frontend, backend, and database responsibilities simplifies maintenance because updates can be applied to one layer without requiring significant modifications to the others. Secondly, scalability is improved since backend services and database resources can be upgraded independently as the number of users increases. Thirdly, security is strengthened because sensitive operations such as authentication, authorization, and data validation are performed on the server rather than on the client device. Finally, the architecture supports future extensibility by allowing new application modules and services to be integrated with minimal disruption to existing functionality.

The overall architecture adopted for the implementation of the ACE mobile application is illustrated in Figure 4.1.

Figure 4.1: Overall System Architecture of the ACE Mobile Application

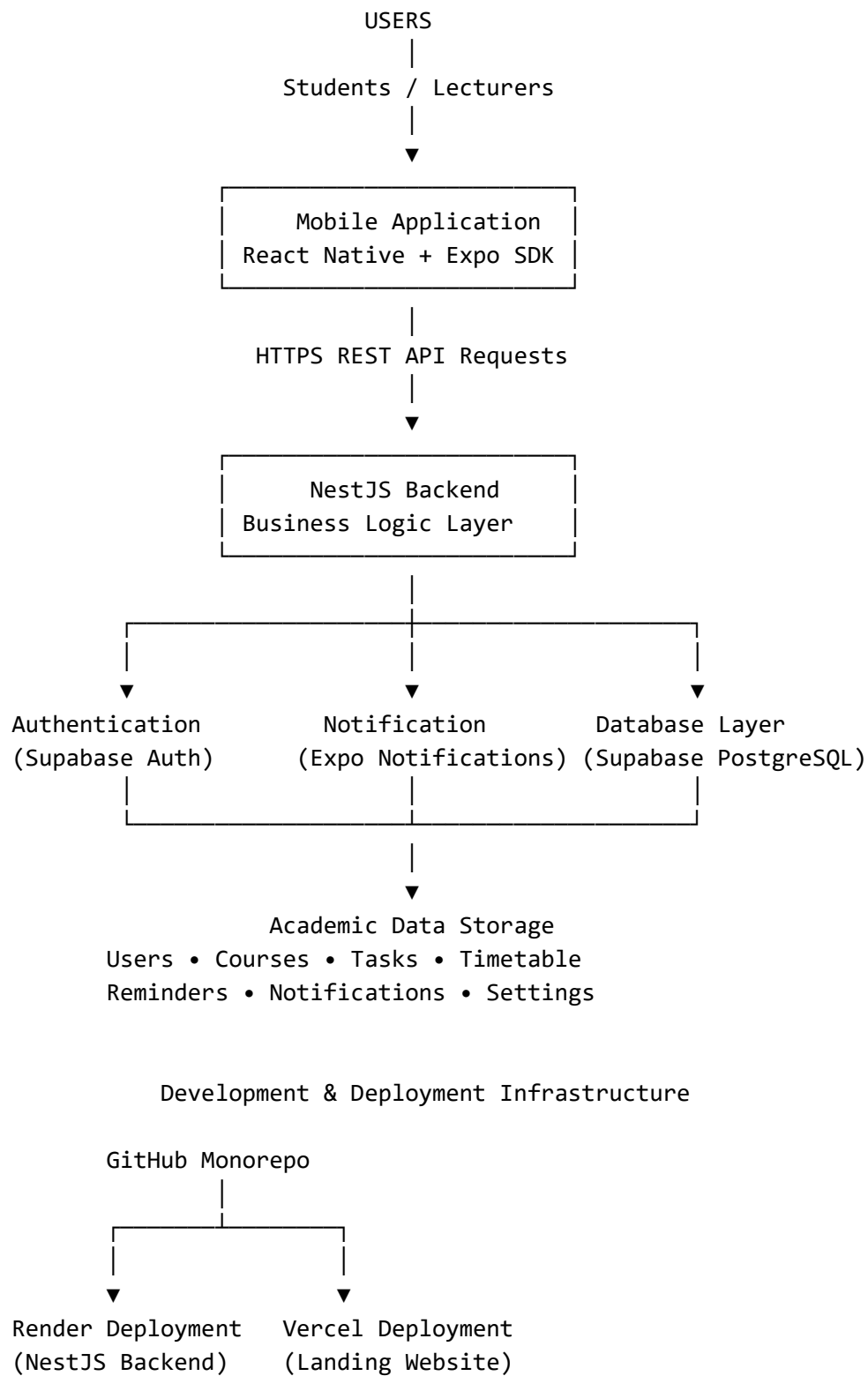


Figure 4.2 Deployment Architecture

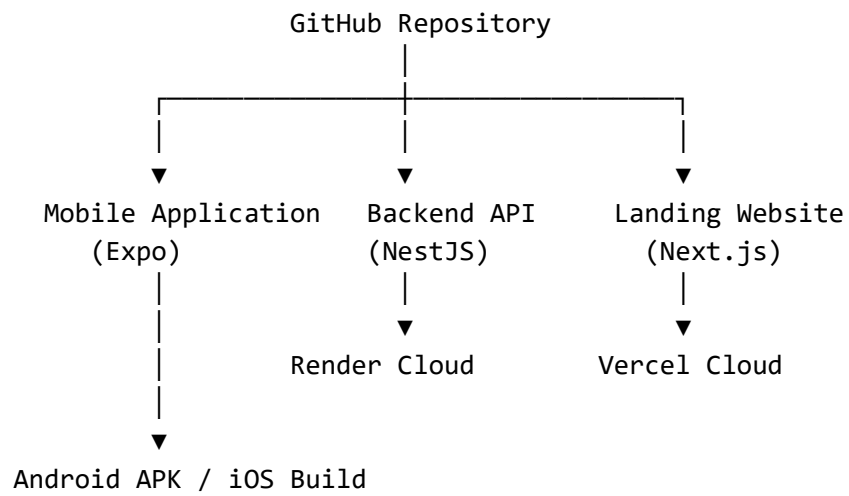
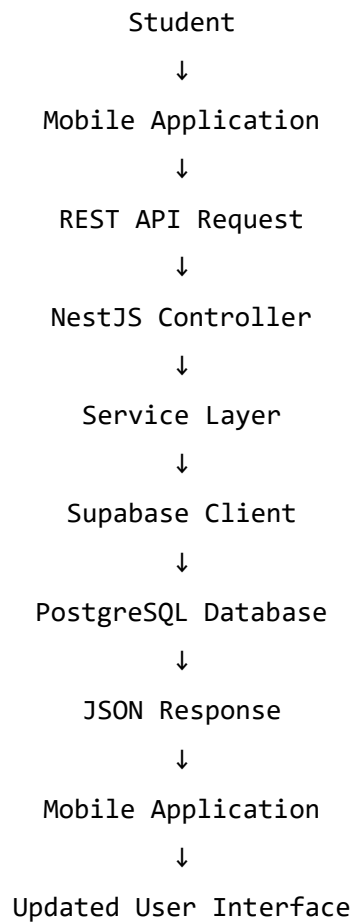


Figure 4.3 Communication Flow



The architecture shown in Figure 4.1 illustrates the interaction between the mobile application, backend server, cloud database, notification services, and web application. It also demonstrates the communication flow from the user interface to the backend services and finally to the database, illustrating how requests and responses are exchanged throughout the system.

4.3.1 Overall Architectural Design

The architectural design of the ACE application follows the client-server software architecture model. Client-server architecture is one of the most widely adopted architectural patterns in modern distributed systems because it separates user-facing applications from server-side processing and centralized data management (Sommerville, 2021).

Within this model, the client is responsible for presenting information to users and collecting user input, while the server processes incoming requests, executes business logic, communicates with the database, and returns appropriate responses.

In the context of the ACE application, the architecture consists of five major components:

i. Mobile Client Application

The mobile application serves as the primary interface through which students interact with the system. It provides features such as user registration, authentication, timetable management, assignment tracking, reminders, notifications, profile management, and settings. The application was developed using React Native and the Expo framework to support both Android and iOS platforms from a single codebase.

The client communicates exclusively with the backend through secure HTTPS requests. No direct communication occurs between the mobile application and the database, thereby improving system security and ensuring that all requests pass through the application's business logic layer.

ii. Backend Application Server

The backend serves as the central processing unit of the system. Developed using the NestJS framework, it implements the business rules governing the application, validates incoming requests, authenticates users, communicates with cloud services, and manages database operations.

The backend exposes RESTful API endpoints that receive requests from the mobile application, process the requested operations, and return standardized JSON responses. By centralizing business logic within the backend, consistency is maintained across all application modules while reducing code duplication and improving maintainability.

iii. Cloud Database

Persistent data generated by users is stored within a PostgreSQL relational database hosted on the Supabase cloud platform. The database stores information relating to user accounts, academic schedules, reminders, tasks, notifications, application settings, and authentication records.

The relational nature of PostgreSQL enables efficient organization of related data while enforcing integrity through primary keys, foreign keys, indexing, constraints, and transactional consistency. Supabase additionally provides secure authentication services, row-level security policies, and managed database infrastructure, reducing administrative overhead while maintaining enterprise-grade security.

iv. Notification Service

One of the major objectives of the ACE application is to remind students about upcoming academic activities. To achieve this objective, the system integrates Expo Notifications, which provides a unified interface for scheduling and delivering local and push notifications across supported mobile platforms.

Notification requests originate from user-created schedules and reminders. The application calculates notification schedules based on the configured reminder settings before delivering alerts to the user's device at the appropriate time.

This notification subsystem significantly enhances the application's usefulness by helping students remain aware of important academic events, deadlines, and scheduled learning activities.

v. Web Landing Platform

In addition to the mobile application, a lightweight web landing page was developed to provide information about the ACE application and facilitate user acquisition. The website introduces the application's objectives, highlights its major features, presents screenshots, and provides download instructions for prospective users.

The landing website also serves as the public-facing component of the project, separating marketing and informational content from the functional mobile application while allowing future expansion through additional web services if required.

4.3.2 Architectural Design Principles

The implementation of the ACE application was guided by several well-established software engineering principles that promote software quality, maintainability, scalability, and long-term sustainability. Rather than developing the system as a tightly coupled application, the architecture was intentionally designed to encourage loose coupling, modularity, code reusability, and clear separation of responsibilities.

The principal architectural design principles adopted during implementation include:

Separation of Concerns (SoC)

Separation of Concerns is a software engineering principle that advocates dividing a software system into independent modules, each responsible for a single aspect of the application's functionality (Pressman & Maxim, 2020). In the ACE application, the presentation layer, business logic layer, and data layer were implemented independently. This separation simplifies maintenance, facilitates testing, and allows individual components to evolve without introducing unnecessary dependencies.

Modularity

The system was implemented as a collection of independent yet interconnected modules. Features such as authentication, timetable management, reminders, notifications, profile management, and settings were developed as distinct functional units. This modular approach improves code organization, promotes reuse, and allows future features to be integrated with minimal restructuring of the existing system.

Reusability

Reusable software components were extensively employed throughout the implementation process. User interface components, utility functions, service classes, validation logic, and API communication modules were designed to be shared across multiple sections of the application. This reduced code duplication while improving consistency and simplifying maintenance.

Scalability

The adopted architecture allows the application to accommodate future growth in both functionality and user population. Since the frontend, backend, and database operate independently, each layer can be upgraded or expanded without requiring complete redevelopment of the entire application. Future features such as institutional integration, lecturer portals, artificial intelligence assistants, cloud synchronization, and analytics dashboards can therefore be incorporated within the existing architecture.

Maintainability

Maintainability refers to the ease with which software can be modified, corrected, or enhanced after deployment. Through the adoption of TypeScript, NestJS, modular folder structures, centralized API services, and version control using Git, the implementation of the ACE application promotes clean, organized, and maintainable source code that supports long-term software evolution.

Documentation and Software Sustainability

Software documentation is widely recognized as an essential component of the software development lifecycle. While the source code defines the behaviour of a software application, documentation provides the descriptive information required for developers, maintainers, testers, and future contributors to understand the purpose, structure, implementation, deployment, and operation of the system. According to Sommerville (2021), comprehensive software documentation improves software maintainability, simplifies knowledge transfer, reduces development time for future contributors, and facilitates long-term software evolution.

Throughout the development of the Academic Companion for Education (ACE), documentation was treated as an integral part of the implementation process rather than as a post-development activity. This approach ensured that important implementation decisions, installation procedures, project architecture, development workflows, and deployment configurations were properly documented alongside the application source code. Maintaining documentation throughout development also ensured that future maintenance and feature enhancements could be performed more efficiently.

The project documentation was primarily developed using **Markdown (MD)** files. Markdown is a lightweight markup language introduced by Gruber (2004) for producing structured technical documents using plain text syntax. It has become the de facto documentation standard within modern software engineering because of its readability, simplicity, portability, compatibility with version control systems, and ability to be automatically rendered into HTML by platforms such as GitHub. Unlike conventional word processing documents, Markdown files can be version-controlled alongside application source code, allowing documentation to evolve simultaneously with software implementation.

Several Markdown documents were created during the implementation of the ACE application to describe various aspects of the project. These documents include the primary project **README**, installation guides, deployment instructions, project architecture descriptions, configuration notes, development procedures, feature documentation, and implementation details for different modules of the system. By documenting these processes, developers can understand the organization of the project, reproduce the development environment, deploy the application correctly, and contribute additional features without requiring extensive external guidance.

The **README.md** file serves as the primary entry point into the project's documentation. It provides a comprehensive overview of the application, including its objectives, implemented features, technology stack, installation requirements, project structure, development workflow, deployment procedures, and usage instructions. The README also introduces prospective developers to the organization of the monorepository, enabling them to understand the relationship between the mobile application, backend service, and web interface before modifying the source code.

In addition to the README file, supplementary Markdown documents were used to document implementation decisions, module responsibilities, development milestones, feature specifications, configuration settings, deployment processes, and project updates throughout the software development lifecycle. Maintaining these documents alongside the source code ensured that technical knowledge remained synchronized with implementation changes, thereby reducing inconsistencies between documentation and application behaviour.

The documentation strategy adopted during this study aligns with contemporary DevOps and software engineering practices, where documentation is regarded as a living component of the software project. Rather than producing documentation only after implementation, developers continuously update technical documents as the software evolves. This approach promotes software sustainability by ensuring that future maintenance, debugging, deployment, and feature extension activities can be performed with minimal ambiguity.

Furthermore, project versioning and documentation management were supported through Git and GitHub. Every significant modification to the application source code was accompanied by corresponding documentation updates where necessary, ensuring consistency between implementation and project documentation. The GitHub repository therefore serves not only as the project's version control platform but also as its centralized technical documentation repository.

The primary project documentation can be found within the project's GitHub repository, particularly through the **README.md** file and other accompanying Markdown documentation that describe the implementation and deployment of the application.

4.3.3 Monorepository Architecture

Modern software engineering projects frequently adopt a **monorepository (monorepo)** development strategy in which multiple related applications are maintained within a single version-controlled repository. Unlike the traditional multi-repository approach where each application is stored independently, a monorepository centralizes the frontend, backend, shared resources, documentation, configuration files, and deployment workflows within one unified project structure. According to Google Engineering (Potvin & Levenberg, 2016), monorepositories promote code consistency, improve dependency management, simplify collaboration, and facilitate coordinated software releases across multiple components of a system.

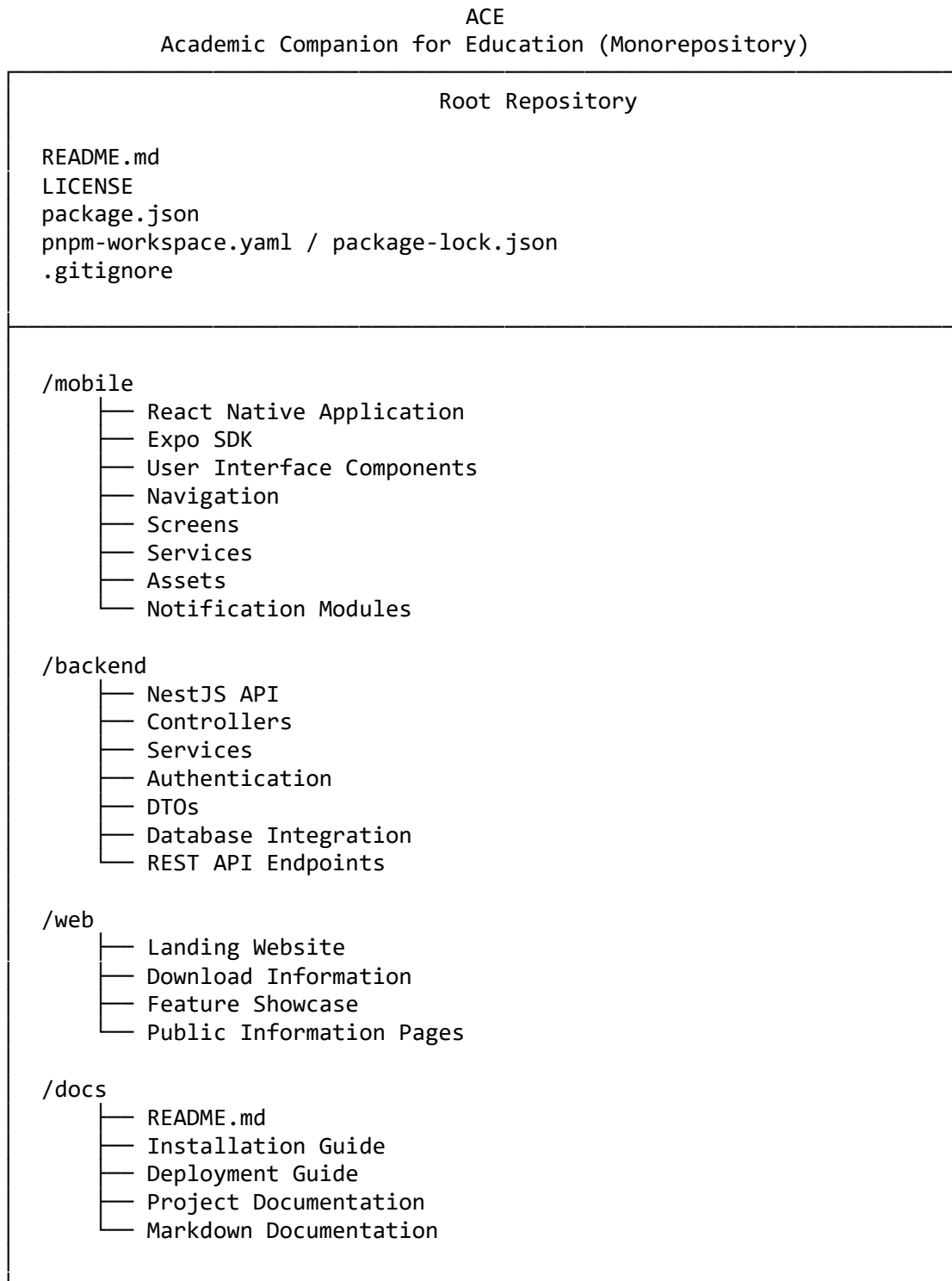
The ACE application was implemented using a monorepository architecture to centralize the development of its three principal software components, namely the mobile application, backend services, and landing website. This architectural approach ensured that all related source code, documentation, configuration files, deployment scripts, and project assets were maintained within a single GitHub repository throughout the development lifecycle.

The major components of the repository include:

- **Mobile Application:** Contains the React Native and Expo source code responsible for the user interface, application navigation, local storage, notification handling, and communication with backend services.
- **Backend Application:** Contains the NestJS server responsible for implementing the application's business logic, authentication, request validation, RESTful API endpoints, database communication, and interaction with third-party services.
- **Web Landing Page:** Contains the website developed to introduce the ACE application to prospective users, provide information regarding its objectives and features, and facilitate access to application downloads.
- **Project Documentation:** Contains Markdown documentation files, including the README, implementation notes, installation guides, deployment instructions, and supporting technical documentation maintained throughout development.

The overall organization of the project repository is illustrated in Figure 4.4.

Figure 4.4: High-Level Monorepository Structure of the Academic Companion for Education (ACE) Project



📁 .github/workflows	add ace web, add mobile and backend	5 days ago
📁 UI Designs	Initial commit	last week
📁 ace	fix: add download button	5 days ago
📁 android	Finally fixed crash and app now opens	1 hour ago
📁 app	add all	5 days ago
📁 assets	Finally fixed crash and app now opens	1 hour ago
📁 backend	more work redone	5 days ago
📁 src	fixed crash	3 days ago
📁 supabase/migrations	add all	5 days ago
📄 .easignore	Finally fixed crash and app now opens	1 hour ago
📄 .gitignore	Finally fixed crash and app now opens	1 hour ago
📄 PROJECT_REPORT.md	Initial commit	last week
📄 README.md	Initial commit	last week
📄 app.json	Finally fixed crash and app now opens	1 hour ago
📄 babel.config.js	Initial commit	last week
📄 eas.json	add ace web, add mobile and backend	5 days ago
📄 errorlog.txt	Finally fixed crash and app now opens	1 hour ago
📄 logo.png	Initial commit	last week
📄 package-lock.json	Finally fixed crash and app now opens	1 hour ago
📄 package.json	Finally fixed crash and app now opens	1 hour ago
📄 tsconfig.json	Initial commit	last week

📖 Readme
🔔 Activity
⭐ 1 star
👁 0 watching
🍴 0 forks

Releases 1
ACE v1.0.0 Latest
1 hour ago

Deployments 6
✓ **Production** 1 hour ago
[+ 5 deployments](#)

Packages
No packages published
[Publish your first package](#)

Contributors 1
 Alade ...

Languages

- TypeScript 86.4%
- PLpgSQL 12.1%
- Kotlin 1.2%
- Other 0.3%

Organizing the project as a monorepository provided several advantages throughout the implementation process. Firstly, all software components shared a unified version history, allowing related changes across the mobile application, backend services, documentation, and web interface to be tracked together within a single Git repository. This simplified software maintenance and reduced the possibility of version inconsistencies that commonly arise when multiple repositories are maintained independently.

Secondly, the monorepository encouraged greater consistency throughout the development process. Common development standards, dependency management, coding conventions, documentation practices, and deployment workflows could be maintained uniformly across the entire project. This contributed significantly to software quality and reduced the overhead associated with synchronizing multiple independent repositories.

Another important advantage was the centralized management of project documentation. Rather than storing technical documents separately from the source code, Markdown documentation files were maintained alongside the implementation. This ensured that updates to the software were

accompanied by corresponding documentation updates, thereby reducing discrepancies between implementation and technical documentation. The documentation includes the primary README.md file, setup guides, deployment instructions, feature descriptions, architectural explanations, and implementation notes that collectively improve software maintainability and facilitate future development.

The adoption of Markdown documentation further aligns with contemporary software engineering practices in which documentation is regarded as a living component of the software rather than a separate deliverable. Because Markdown documents are plain-text files, they can be version-controlled using Git, reviewed through pull requests, and updated alongside source code modifications. This approach supports collaborative development while ensuring that project documentation remains synchronized with software evolution throughout the software development lifecycle.

Furthermore, the monorepository simplified continuous deployment activities. Since the mobile application, backend API, and web landing page originated from a common repository, cloud deployment platforms were able to monitor only the relevant project directories during deployment. The backend application was deployed to Render, while the web landing page was deployed through Vercel. Whenever modifications were committed and pushed to the GitHub repository, the respective deployment platforms automatically rebuilt and redeployed the affected services. This automated deployment pipeline reduced manual intervention, minimized deployment errors, and ensured that production services remained consistent with the latest stable version of the source code.

From a software engineering perspective, the monorepository also promotes software scalability and maintainability. As additional application modules, services, or client platforms are introduced in future versions of the ACE application, they can be integrated into the existing repository without altering the overall project organization. This extensible repository structure therefore supports the long-term evolution of the application while maintaining a coherent development workflow.

4.3.4 Mobile Application Architecture

The mobile application constitutes the presentation layer of the Academic Companion for Education (ACE) system. The presentation layer is responsible for providing an interactive interface through which users communicate with the underlying application services. According to Sommerville (2021), the presentation layer should remain primarily concerned with user interaction, data presentation, and request initiation, while delegating business logic and persistent data management to backend services.

The ACE mobile application was developed using **React Native** through the **Expo Framework**, enabling a single codebase to support both Android and iOS devices. This cross-platform development approach significantly reduced implementation effort while maintaining a native user experience across supported mobile operating systems. The application follows a modular architecture in which individual functionalities are implemented as independent screens, reusable components, services, hooks, and utility modules. This architectural organization improves code maintainability, facilitates component reuse, and simplifies future software enhancements.

At runtime, user interactions originate from the graphical user interface and are processed through navigation components before invoking application services responsible for communicating with the backend API. Responses received from the backend are subsequently processed and rendered within the user interface, ensuring that users receive real-time updates corresponding to their requests.

The internal architecture of the mobile application is illustrated in Figure 4.5.

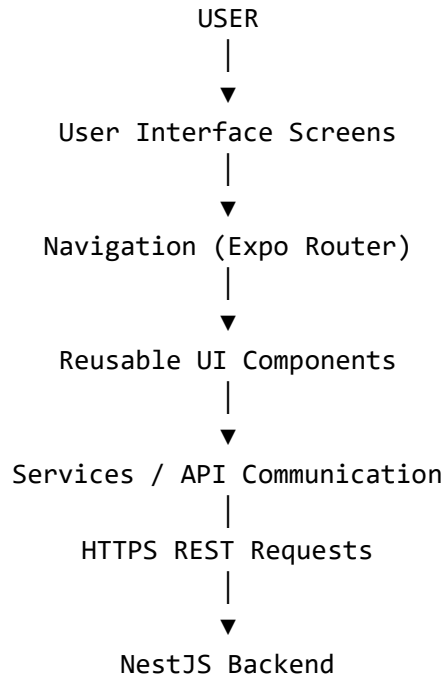


Figure 4.5: Internal Architecture of the ACE Mobile Application

4.3.4 Mobile Application Architecture (Continued)

As illustrated in Figure 4.5, the mobile application follows a layered architectural pattern in which responsibilities are distributed across multiple independent components. This organization adheres to the principle of **Separation of Concerns (SoC)** by ensuring that each layer performs a specific responsibility without unnecessarily depending on other layers. Such separation improves software readability, maintainability, scalability, and testing while reducing code duplication.

The **User Interface Layer** represents the first point of interaction between users and the application. It consists of all screens, pages, forms, buttons, cards, dialogs, icons, and navigation elements presented to students during application usage. These interfaces were first designed and prototyped using Figma before implementation commenced. Developing the interface prototypes prior to coding enabled iterative refinement of the application's layout, navigation flow, colour consistency, typography, spacing, accessibility, and overall user experience. Once the interface designs had been validated, they were implemented using reusable React Native components within the Expo framework.

Application navigation was implemented using the **Expo Router**, a file-based routing system built on top of React Navigation. File-based routing automatically maps application screens to their corresponding navigation routes based on the project's directory structure. This approach simplifies navigation management by reducing manual route configuration while maintaining a logical organization of application pages. The use of Expo Router also facilitates deep linking, nested navigation, and scalable application organization as additional screens are introduced.

Beneath the navigation layer lies the **Component Layer**, which contains reusable interface components utilized across multiple sections of the application. Examples include custom buttons, input fields, cards, modal dialogs, loading indicators, notification widgets, timetable cards, task items, and profile elements. Rather than implementing identical interface elements repeatedly, reusable components were created to promote consistency, reduce code redundancy, and simplify future maintenance.

The **Service Layer** acts as the communication bridge between the mobile application and the backend server. Rather than allowing interface screens to communicate directly with external services, dedicated service modules were implemented to encapsulate all API communication. These services are responsible for sending HTTP requests, receiving server responses, handling authentication tokens, processing errors, and converting server responses into application objects suitable for presentation within the user interface.

Whenever a user performs an operation, such as logging into the application, creating a reminder, updating profile information, or retrieving timetable entries, the request is forwarded to the appropriate service module. The service subsequently communicates with the NestJS backend through secure HTTPS requests using RESTful APIs. Once the backend completes the requested operation, the response is returned to the mobile application, where the interface is updated accordingly.

The layered organization adopted throughout the mobile application significantly improves maintainability because modifications to one layer rarely require changes to other layers. For example, adjustments to the user interface do not affect backend communication logic, while modifications to API endpoints require updates only within the service layer rather than throughout

the application. This architectural approach therefore promotes cleaner software organization and supports future system expansion.

4.3.5 Backend Architecture

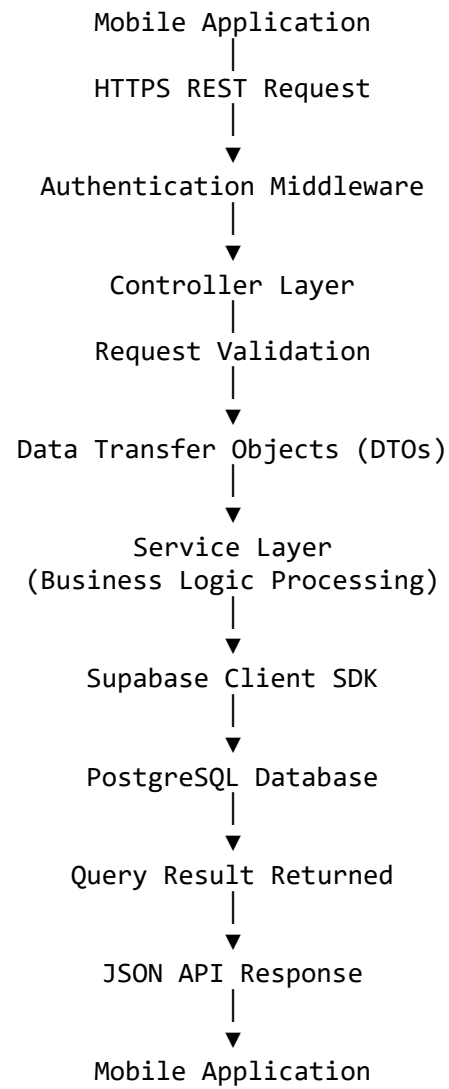
The backend architecture constitutes the application layer of the ACE system and serves as the intermediary between the mobile application and the cloud database. While the frontend is responsible for presenting information to users, the backend manages application logic, validates incoming requests, enforces security policies, performs authentication, communicates with the database, and returns appropriate responses to client applications.

The backend was implemented using **NestJS**, a progressive Node.js framework developed with TypeScript and inspired by Angular's modular architectural principles. NestJS promotes the development of scalable and maintainable server-side applications through dependency injection, modular programming, decorators, and layered application design (NestJS, 2025). These architectural features make NestJS particularly suitable for enterprise-level RESTful web services and cloud-based applications.

The backend implementation of the ACE application follows a modular layered architecture consisting primarily of controllers, services, data transfer objects, authentication mechanisms, and database integration modules. Each layer performs clearly defined responsibilities while interacting with adjacent layers through standardized interfaces.

The architecture adopted by the backend is illustrated in Figure 4.6.

Figure 4.6: Backend Request Processing Architecture



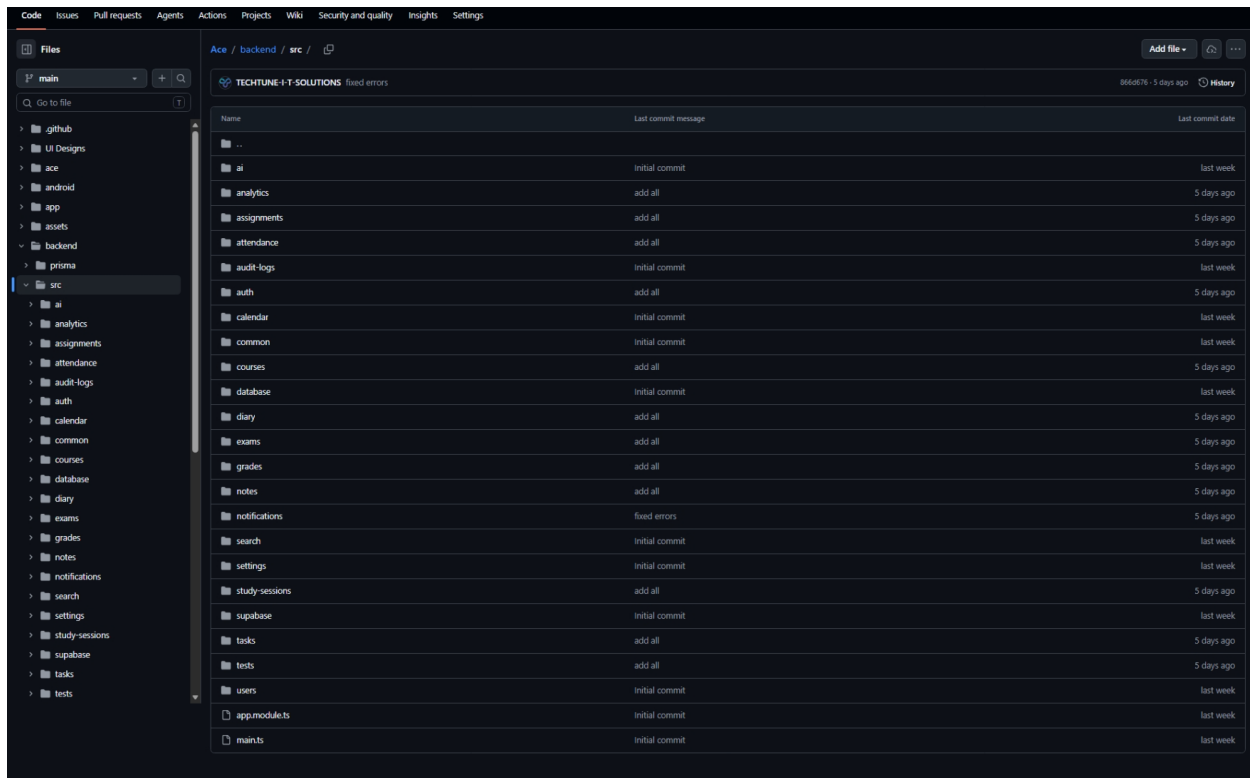


Figure 4.6: Backend Request Processing Workflow

Incoming requests from the mobile application are first received through secure HTTPS endpoints exposed by the backend. Before any business operation is executed, authentication middleware verifies the identity of the requesting user and determines whether sufficient authorization exists to perform the requested action. This verification process contributes significantly to application security by preventing unauthorized access to protected resources.

Following successful authentication, requests are forwarded to the appropriate **Controller**. Controllers represent the entry point of the backend application and are responsible for receiving incoming HTTP requests, extracting request parameters, invoking the required service methods, and returning standardized HTTP responses. Controllers themselves contain minimal business logic, thereby maintaining separation between request handling and application processing.

To improve data integrity, incoming requests undergo validation through **Data Transfer Objects (DTOs)**. DTOs define the expected structure of request data and verify that submitted information satisfies predefined validation rules before processing continues. This mechanism prevents invalid

or incomplete data from entering the application's business logic, thereby reducing runtime errors and improving application reliability.

Once validation has been completed successfully, processing is transferred to the **Service Layer**. The service layer contains the core business logic of the application and implements the functional requirements described in earlier chapters. Services manage user authentication, timetable operations, reminder scheduling, profile management, task handling, notification processing, and communication with external cloud services.

The service layer subsequently communicates with the Supabase client library, which serves as the intermediary between the backend application and the PostgreSQL database. Database operations including record insertion, retrieval, updating, deletion, filtering, and authentication are performed through secure database transactions. Query results returned from PostgreSQL are processed by the backend before being converted into standardized JSON responses and transmitted back to the mobile application.

This layered backend architecture provides several software engineering advantages. Firstly, it promotes modularity by separating request handling from business logic and database access. Secondly, it improves maintainability by allowing modifications to individual services without affecting unrelated application modules. Thirdly, centralized validation enhances application security and data integrity. Finally, the architecture supports future scalability by enabling additional modules to be incorporated into the backend with minimal structural modification.

4.3.6 Database Architecture

Persistent data management forms one of the most critical components of any information system because it ensures that user-generated information remains available across application sessions. A database provides structured mechanisms for storing, organizing, retrieving, updating, and protecting data while maintaining consistency and integrity. According to Elmasri and Navathe (2017), relational database management systems organize information into related tables connected through keys and constraints, thereby minimizing redundancy and ensuring accurate data relationships.

The ACE application employs **Supabase**, a cloud-based Backend-as-a-Service (BaaS) platform, as its primary database infrastructure. Supabase utilizes **PostgreSQL**, one of the world's most widely adopted open-source relational database management systems, providing enterprise-grade reliability, transaction support, indexing, relational modelling, authentication services, and fine-grained access control.

Unlike local databases embedded within mobile applications, the cloud-based database architecture adopted in this study enables centralized storage of user information while supporting secure access from multiple authenticated devices. This approach ensures that user records remain synchronized, protected, and recoverable even when devices are replaced or applications are reinstalled.

The logical organization of the application's database is presented in Figure 4.7.

Figure 4.7: Simplified Database Entity Relationship Diagram (ERD)

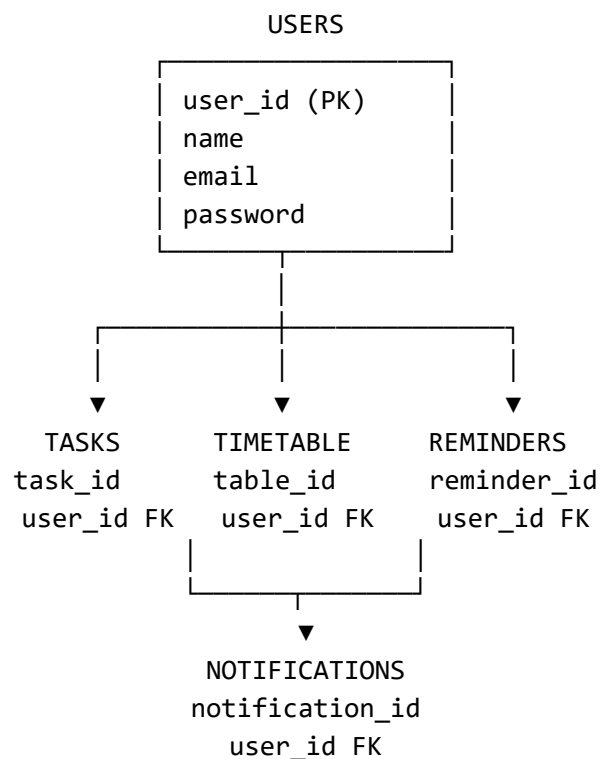


Figure 4.7: Simplified Entity Relationship Diagram of the ACE Database

The relational structure illustrated in Figure 4.7 demonstrates how user information forms the central entity upon which other academic records depend. Primary keys uniquely identify each record, while foreign keys establish relationships between users and their associated tasks, timetables, reminders, and notifications. This relational organization minimizes data redundancy and preserves referential integrity throughout the database.

4.3.7 Functional Module Architecture

One of the major software engineering principles adopted during the implementation of the Academic Companion for Education (ACE) was **modular software architecture**. Rather than implementing all application functionalities within a single source file or software component, the application was organized into independent functional modules, each responsible for a specific aspect of the academic management process. According to Sommerville (2021), modular software architecture promotes maintainability, reusability, scalability, and easier testing by dividing complex software systems into manageable and loosely coupled components.

The backend implementation of the ACE application follows this modular design philosophy through the organization of independent feature modules within the NestJS framework. Each module encapsulates its own controllers, services, data models, validation mechanisms, and business logic while communicating with other modules through clearly defined interfaces. This architectural organization minimizes coupling between unrelated components and simplifies future software enhancements.

Figure 4.8 illustrates the major functional modules implemented within the backend application.

Figure 4.8: Functional Module Organization of the ACE Backend

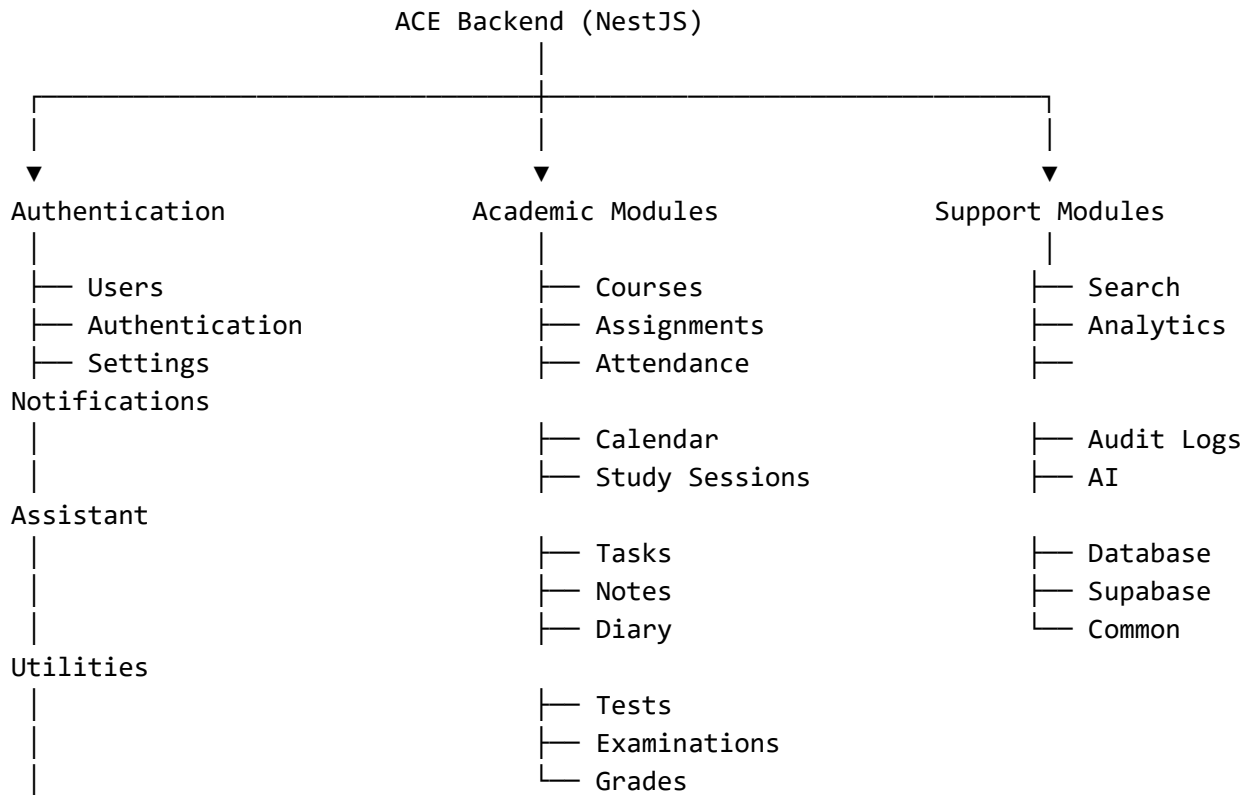


Figure 4.8: Functional Organization of Backend Modules

The modular organization illustrated in Figure 4.8 demonstrates the logical grouping of related functionalities within the ACE backend. Rather than centralizing all academic operations within a single service, individual modules were developed to manage specific domains of the application. This approach promotes separation of concerns, allowing each module to evolve independently while maintaining interoperability with the remainder of the system.

The major modules implemented during the development of the application are described as follows.

4.4 System Module Implementation

4.4.1 Authentication Module

Authentication is one of the most critical components of modern information systems because it establishes the identity of users before granting access to protected resources. The authentication

module developed for the ACE application is responsible for user registration, secure login, account verification, password management, session validation, and authorization.

The module communicates with Supabase Authentication services to verify user credentials while ensuring that only authenticated users can access academic resources stored within the application. Validation procedures were implemented to verify user input before authentication requests are processed, thereby improving application security and minimizing invalid authentication attempts.

4.4.1.1 University of Ilorin Academic Profile Configuration

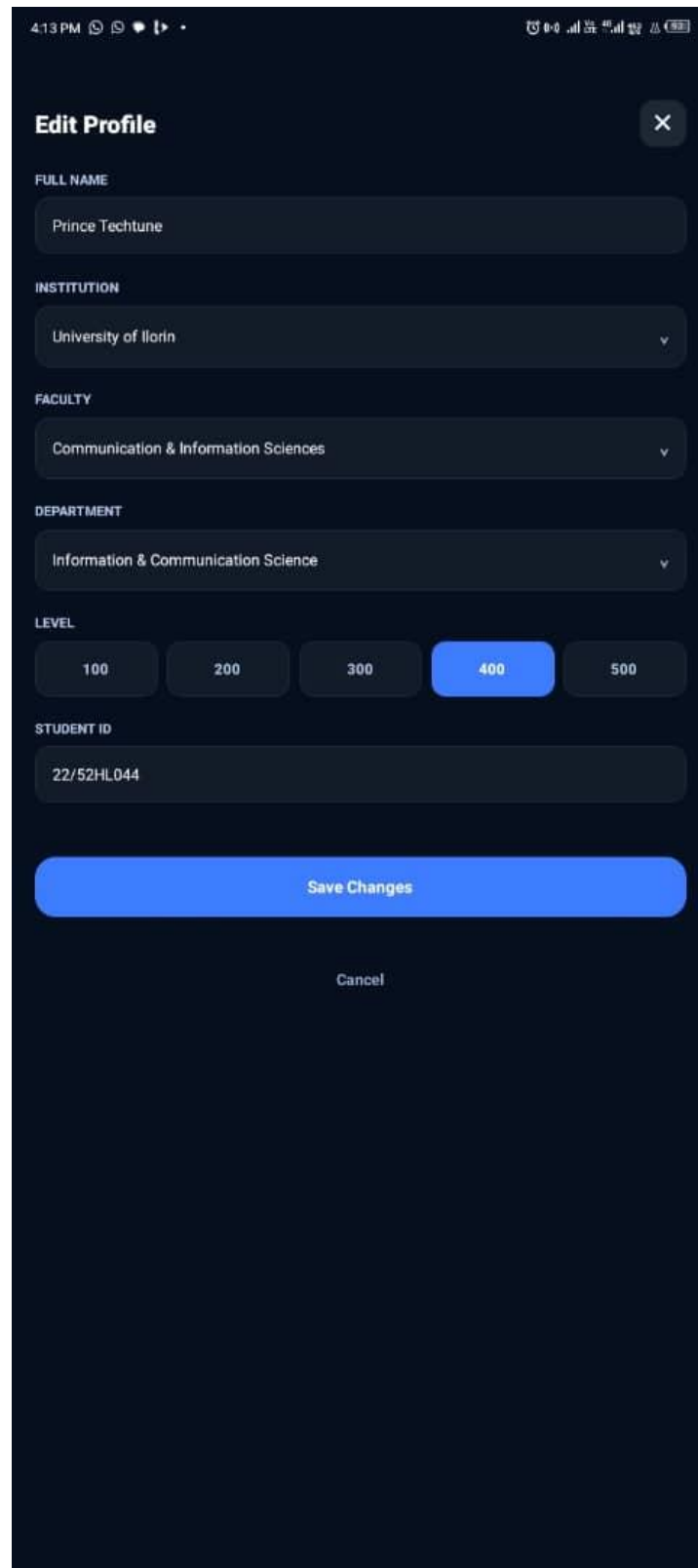
Following successful account registration, newly registered users are required to complete their academic profile by providing institutional information specific to the University of Ilorin. This process enables the application to personalize academic scheduling and reminder services according to each student's educational context.

During profile setup, users select their faculty, department, level of study, and course of study from predefined institutional records maintained within the application's database. To support this functionality, the faculties and departments of the University of Ilorin were seeded into the database during system implementation. This approach ensures consistency in academic information, minimizes user input errors, and prevents the creation of duplicate or invalid academic records.

The academic profile subsequently serves as contextual information throughout the application. Student-specific information may be utilized for organizing academic activities, grouping courses, generating personalized reminders, supporting future timetable integration, and enabling institution-specific enhancements without requiring users to repeatedly enter their academic details.

The adoption of structured institutional data further aligns the developed system with the objectives of this study by tailoring the application specifically to the academic structure of the University of Ilorin.

Figure 4.9.1: Academic Profile Setup Screen



The image shows a mobile application screen titled "Edit Profile" with a close button (X) in the top right corner. The screen is set against a dark blue background. The form contains the following fields and options:

- FULL NAME:** A text input field containing "Prince Techtune".
- INSTITUTION:** A dropdown menu showing "University of Ilorin" with a downward arrow.
- FACULTY:** A dropdown menu showing "Communication & Information Sciences" with a downward arrow.
- DEPARTMENT:** A dropdown menu showing "Information & Communication Science" with a downward arrow.
- LEVEL:** A row of five buttons labeled "100", "200", "300", "400", and "500". The "400" button is highlighted in blue.
- STUDENT ID:** A text input field containing "22/52HL044".

At the bottom of the form, there is a large blue button labeled "Save Changes" and a smaller, faint "Cancel" button below it.

4.4.2 User Management Module

The User Management module maintains personal information associated with each registered student. It manages profile details, account preferences, application personalization settings, and user-specific academic information required throughout the system.

Because every academic record is associated with an individual user, this module serves as the foundation upon which other functional modules operate. User identifiers generated during registration establish relationships with schedules, assignments, attendance records, notes, grades, reminders, and other academic information stored within the database.

4.4.3 Course Management Module

The Course Management module enables students to organize and maintain information relating to registered courses. Academic courses constitute the primary organizational structure around which assignments, examinations, attendance records, grades, and study sessions are managed.

Unlike generic note-taking applications, the Courses module was designed to reflect the academic structure of the University of Ilorin. Students are able to create and organize individual courses corresponding to the courses they register during each academic session. Course information subsequently serves as the foundation upon which assignments, examinations, attendance records, study sessions, notes, and reminders are organized within the application.

This hierarchical organization allows students to manage their academic activities on a course-by-course basis, thereby reducing information fragmentation while improving academic planning and retrieval of course-related records.

Each course record contains descriptive information that enables other academic modules to associate educational activities with the appropriate subject or course of study. Centralizing course information simplifies academic organization and improves consistency across the application.

4.4.4 Assignment Management Module

The Assignment module allows students to record coursework, monitor submission deadlines, update completion status, and maintain organized records of academic tasks assigned by lecturers. Each assignment can be linked to a specific course while supporting reminder scheduling through the notification subsystem.

This module contributes significantly to the objective of reducing missed deadlines by providing centralized assignment tracking integrated with the application's scheduling capabilities.

4.4.5 Attendance Management Module

Attendance monitoring forms an important aspect of academic performance within tertiary institutions. The Attendance module enables students to record lecture attendance, monitor attendance percentages, and maintain historical attendance records for individual courses.

The information generated by this module may subsequently be analyzed alongside examination performance and course completion statistics to provide students with a clearer understanding of their academic engagement.

4.4.6 Examination and Test Modules

The Examination and Test modules were implemented to assist students in organizing continuous assessments, quizzes, practical examinations, and final examinations throughout the academic session. These modules maintain schedules, examination dates, assessment records, and associated reminders.

Through integration with the calendar and notification modules, students receive timely reminders regarding upcoming assessments, thereby improving preparedness and reducing the likelihood of missed examinations.

4.4.7 Grade Management Module

The Grade module enables students to record, organize, and review academic performance across multiple courses. Assessment scores obtained from tests, assignments, practical exercises, and examinations may be maintained within the application to provide students with a centralized record of academic progress.

Further analytical components may utilize these records to generate performance summaries and identify areas requiring academic improvement.

4.4.8 Calendar and Scheduling Module

The Calendar module provides chronological organization of academic activities occurring throughout the semester. Lectures, examinations, assignment deadlines, study sessions, and personal academic events are consolidated within a unified calendar interface, allowing students to visualize upcoming responsibilities more effectively.

Integration with reminder and notification services enables scheduled events to generate timely alerts before their occurrence.

4.4.9 Task Management Module

Task management is an important aspect of academic productivity because it enables students to organize daily responsibilities, prioritize activities, and monitor the completion of academic obligations. Effective task management contributes to improved time management, reduced procrastination, and increased academic performance by ensuring that important activities are completed within their scheduled timeframes (Allen, 2015).

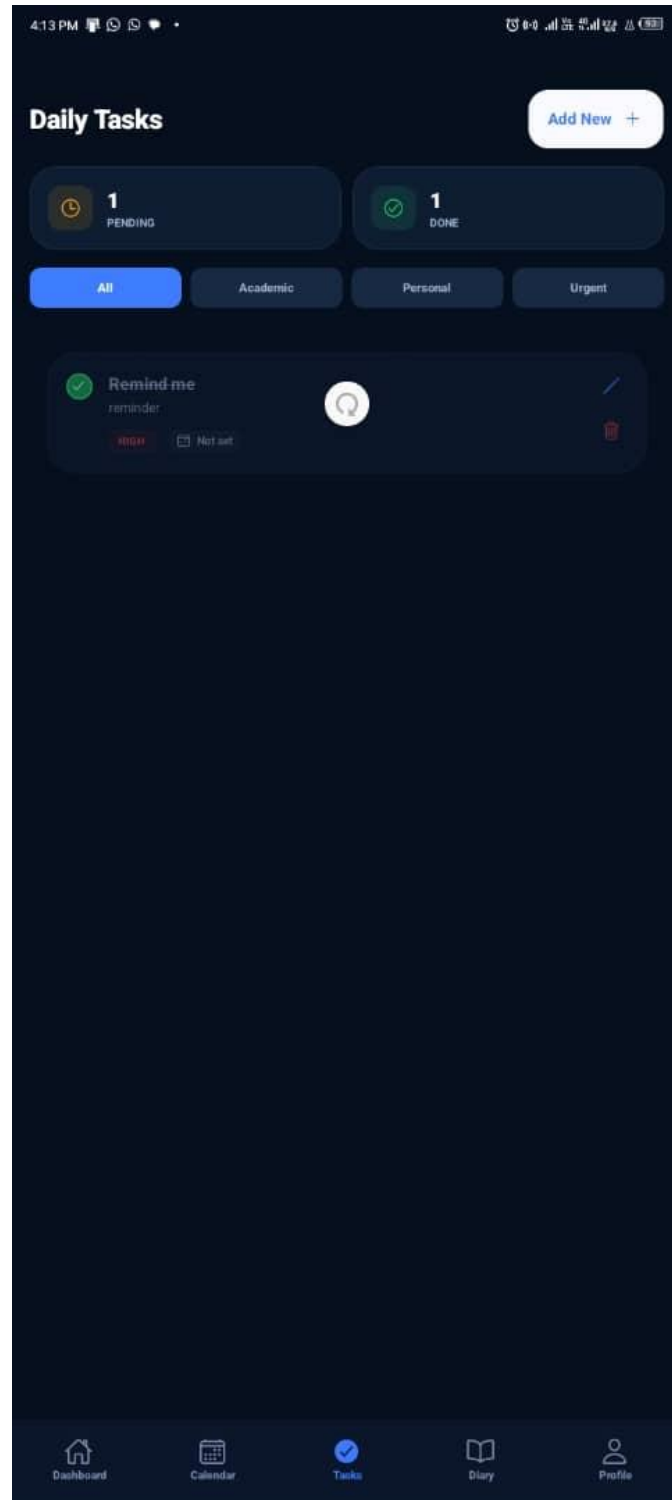
The Task Management module of the ACE application was developed to provide students with a centralized platform for managing both academic and personal tasks. Unlike assignment records, which are directly associated with specific courses, the task management system allows students to create general-purpose tasks that may relate to lectures, projects, meetings, extracurricular activities, personal goals, or other academic responsibilities.

Each task contains relevant information such as the task title, description, priority level, due date, completion status, and creation timestamp. The backend validates all incoming task requests before storing them within the PostgreSQL database, ensuring that only valid records are maintained.

The module communicates closely with the notification subsystem, allowing reminders to be scheduled automatically for tasks approaching their deadlines. Once a task has been completed, students may update its status, enabling the dashboard and analytics modules to provide more accurate summaries of pending and completed activities.

By maintaining an organized record of pending responsibilities, the task management module assists students in prioritizing their workload and improving academic organization.

Figure 4.9.2: Task Management Interface



4.4.10 Notes Management Module

Academic note-taking plays an important role in knowledge retention, revision, and long-term learning. Students frequently record lecture summaries, research findings, important announcements, revision materials, and personal observations throughout the academic semester.

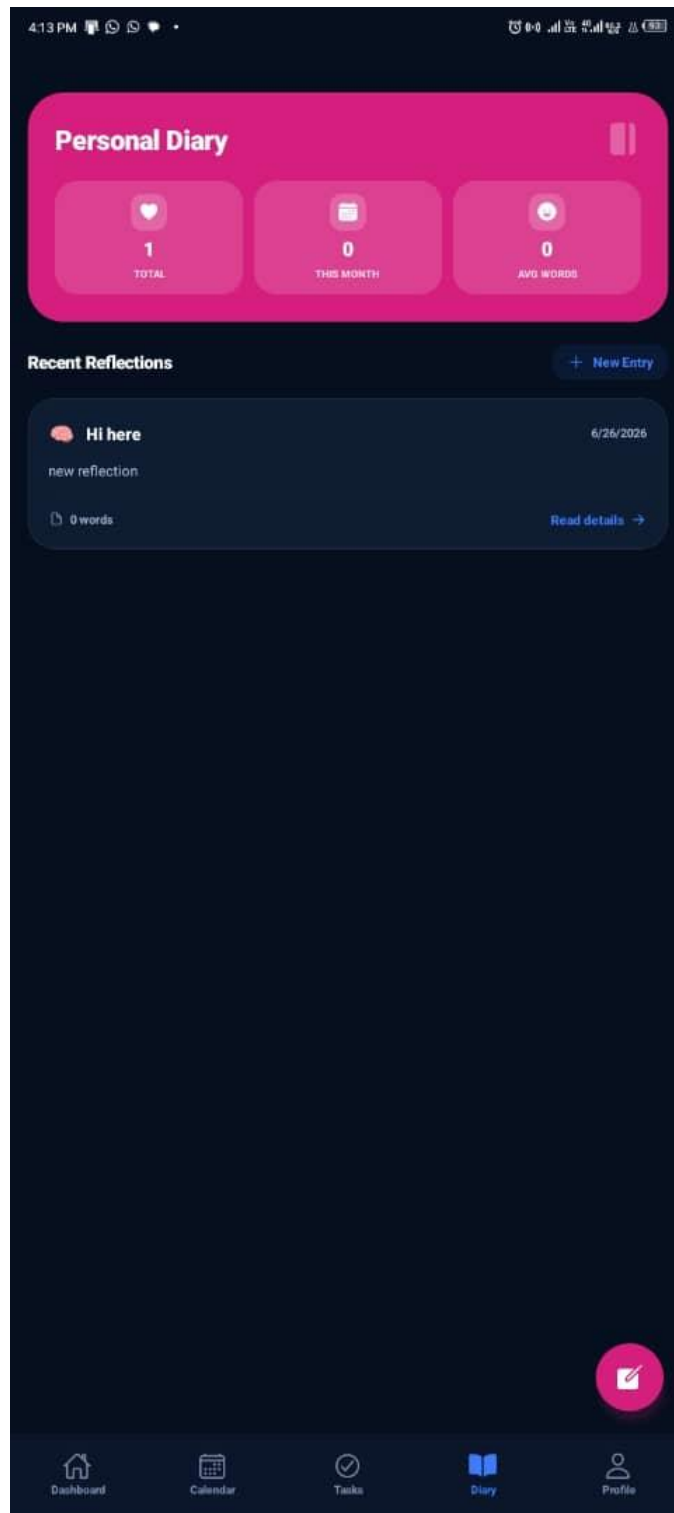
The Notes Management module was developed to provide a secure and organized environment for storing academic notes directly within the application. Rather than relying on external note-taking applications, students can create, edit, categorize, search, and delete notes while maintaining synchronization with their academic records.

Each note consists of a title, content, creation date, modification date, and ownership information associated with the authenticated user. Notes are securely stored within the cloud database, ensuring that they remain available across devices following user authentication.

To improve usability, the module integrates with the application's search functionality, enabling students to retrieve stored notes efficiently without manually browsing through large collections of information.

The Notes module therefore serves as a digital notebook that complements other academic management features implemented within the application.

Figure 4.10: Notes Management Interface



4.4.11 Diary Module

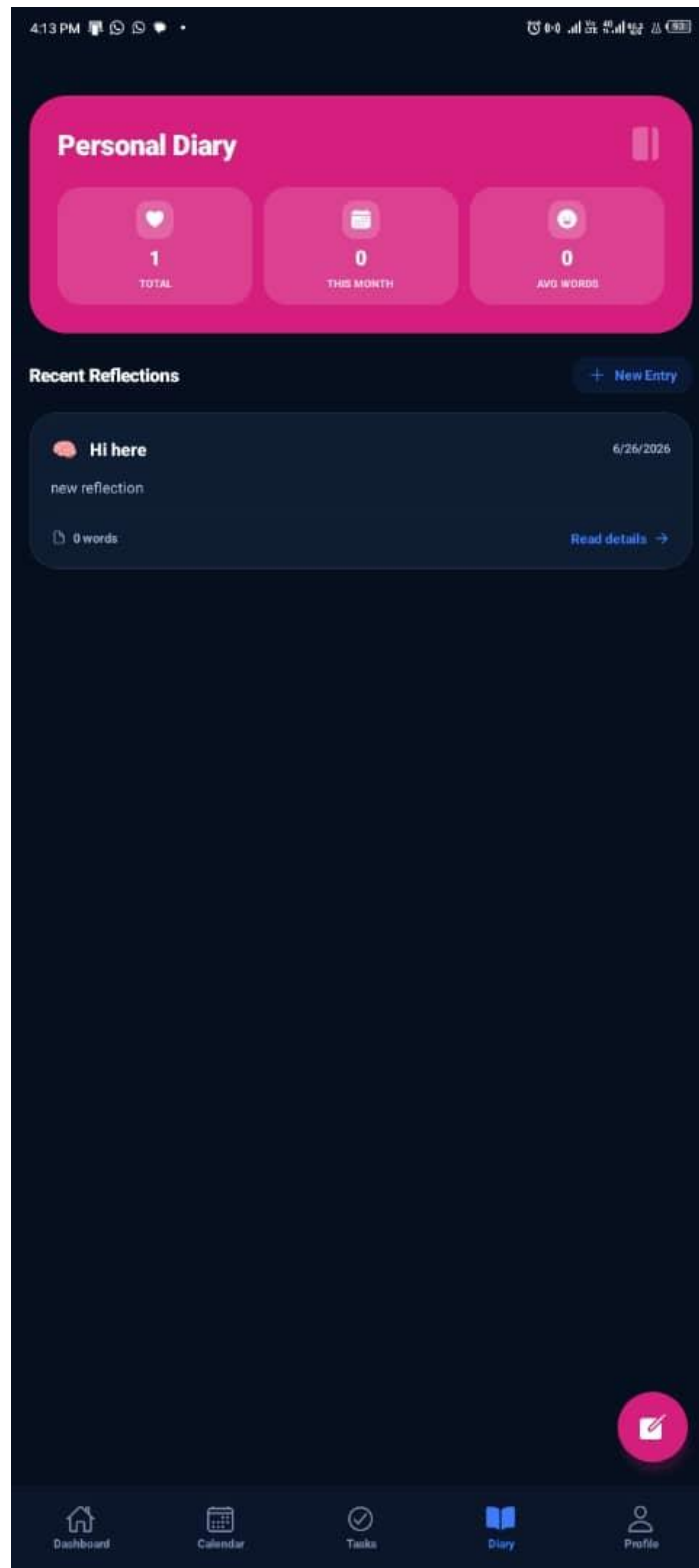
Maintaining a personal academic diary encourages students to reflect on their daily learning experiences, record important observations, and monitor personal development throughout the academic session. Educational researchers have shown that reflective learning promotes deeper understanding, self-awareness, and continuous improvement.

The Diary module provides students with a private environment for documenting daily academic experiences, personal reflections, learning challenges, achievements, and future goals. Unlike the Notes module, which focuses primarily on academic content, the diary emphasizes personal reflection and self-evaluation.

Diary entries are securely associated with individual user accounts, ensuring that only authenticated users can access their personal records. Students may create new entries, update previous records, review historical entries, and remove obsolete information when necessary.

This module contributes to the overall objective of the ACE application by encouraging reflective learning and supporting students' academic development beyond conventional task management.

Figure 4.11: Academic Diary Interface



4.4.12 Study Session Module

Effective study planning is widely recognized as one of the most important contributors to academic success. Organized study schedules assist students in maintaining consistent learning habits, improving concentration, and reducing examination-related stress.

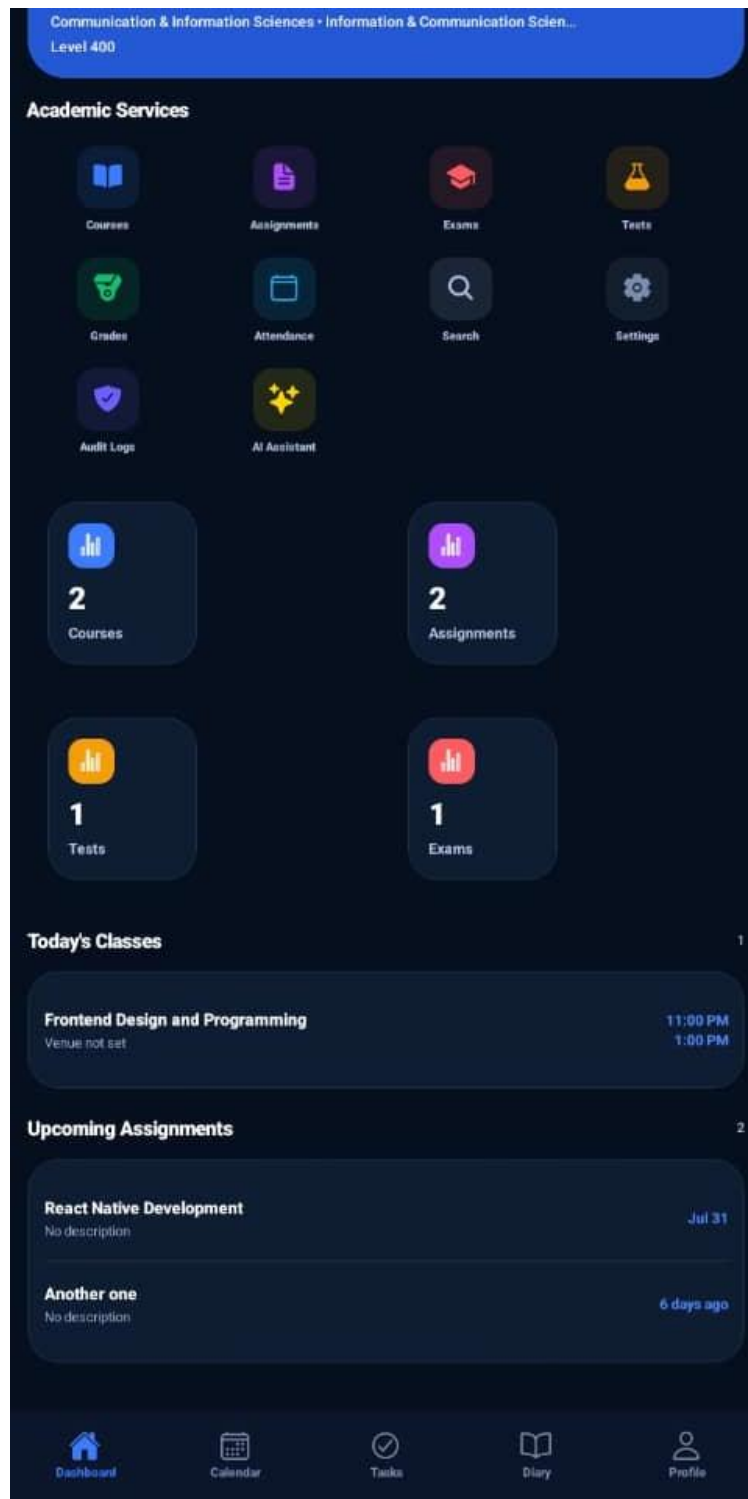
The Study Session module enables students to schedule structured learning periods for individual courses or topics. Each study session includes information such as the associated course, study topic, scheduled date, planned duration, objectives, and completion status.

The module integrates with both the calendar and notification systems, allowing scheduled study sessions to appear within the academic calendar while automatically generating reminder notifications before each session begins.

The implementation of this module encourages disciplined study habits by assisting students in planning and monitoring their learning activities throughout the semester.

The main dashboard further serves as a personalized academic overview for University of Ilorin students by presenting upcoming academic activities based on the information entered throughout the application. Rather than displaying generic reminders, the dashboard aggregates schedules, assignments, attendance activities, examinations, study sessions, and other academic events into a single interface, thereby providing students with an organized view of their ongoing academic responsibilities.

Figure 4.12: Study Session Management Interface



4.4.13 Calendar Module

The Calendar module functions as the central scheduling component of the ACE application. It consolidates information obtained from assignments, examinations, study sessions, reminders, tasks, and other academic activities into a unified chronological interface.

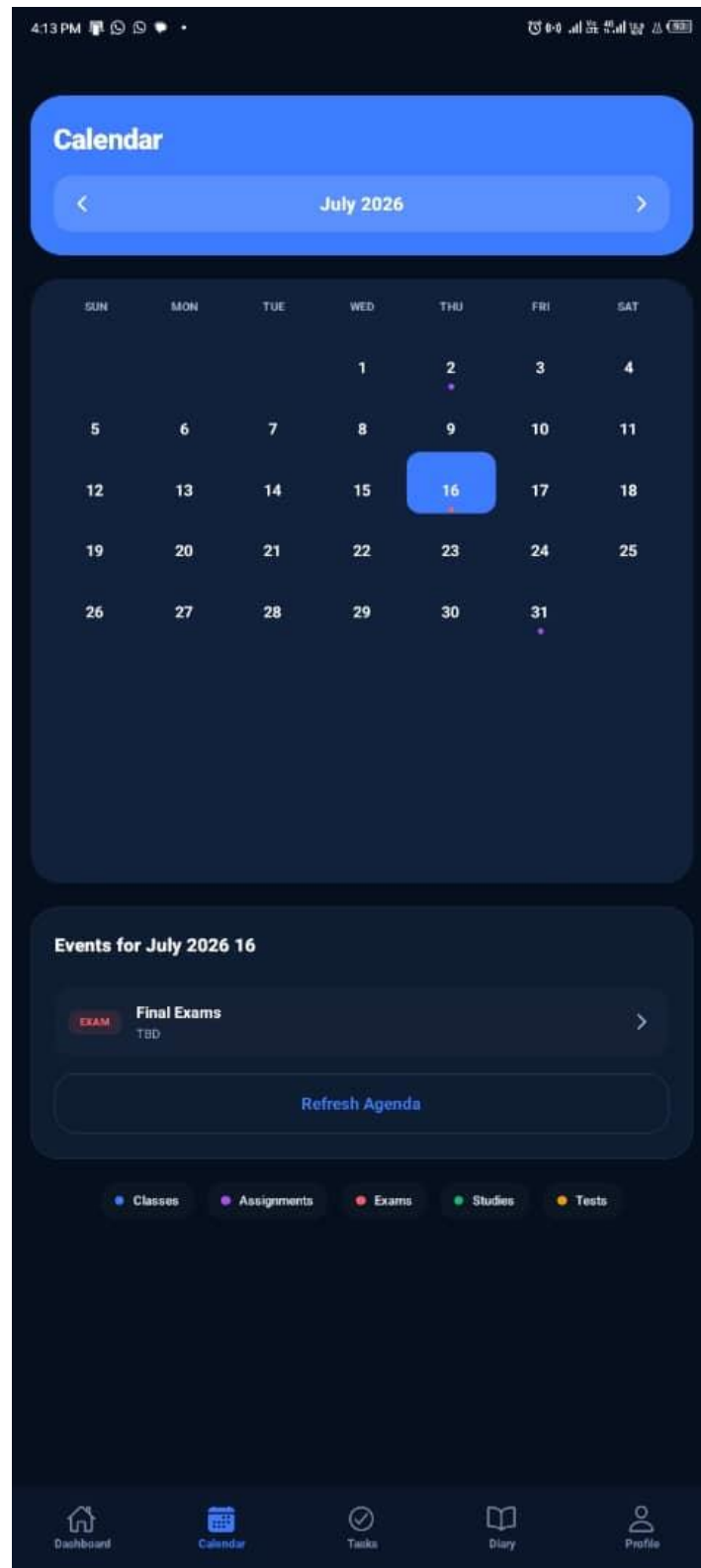
Rather than requiring students to navigate multiple sections of the application to determine upcoming responsibilities, the calendar provides a comprehensive overview of scheduled activities for each day, week, or month.

Calendar events are generated dynamically from information stored within the backend database, ensuring that any updates made to assignments, examinations, or study sessions are immediately reflected within the calendar interface.

The Calendar module directly supports the primary objective of this research by providing students with a centralized environment for planning lectures, assignment deadlines, examinations, study periods, and other academic events. Through visual scheduling and reminder functionality, students are able to organize their academic calendars more efficiently while reducing the likelihood of missed academic activities.

This centralized scheduling approach improves academic planning and assists students in maintaining awareness of upcoming academic obligations.

Figure 4.13: Academic Calendar Interface



4.4.14 Notification Module

One of the principal objectives of the ACE application is to reduce missed academic activities through intelligent reminder mechanisms. To achieve this objective, the application integrates a comprehensive notification subsystem developed using **Expo Notifications** together with backend scheduling services.

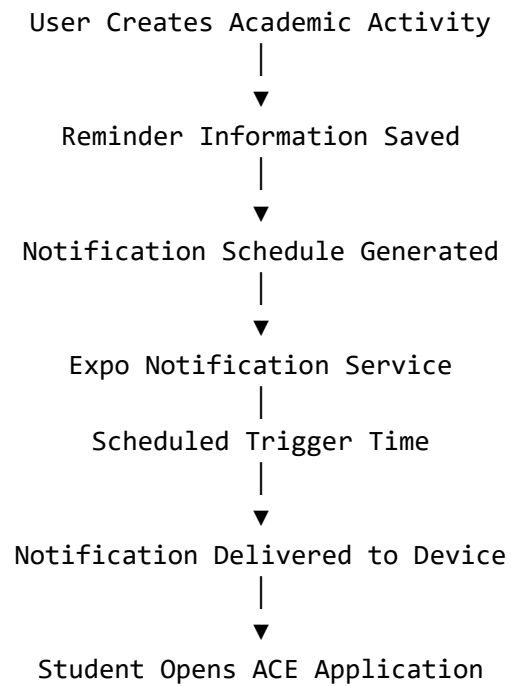
The Notification module manages both locally scheduled notifications and remotely generated application alerts. Whenever a user creates an assignment, examination, task, study session, or reminder with notification enabled, the application calculates the corresponding notification schedule and registers it with the notification service.

The notification subsystem supports various reminder intervals depending on user preferences. Notifications may be scheduled several minutes, hours, or days before the occurrence of an academic event, thereby providing students with sufficient preparation time.

The implementation of push notification services significantly improves the practical usefulness of the application because students continue receiving reminders even when the application is not actively open.

The overall notification process is illustrated in Figure 4.14.

Figure 4.14: Notification Scheduling Workflow



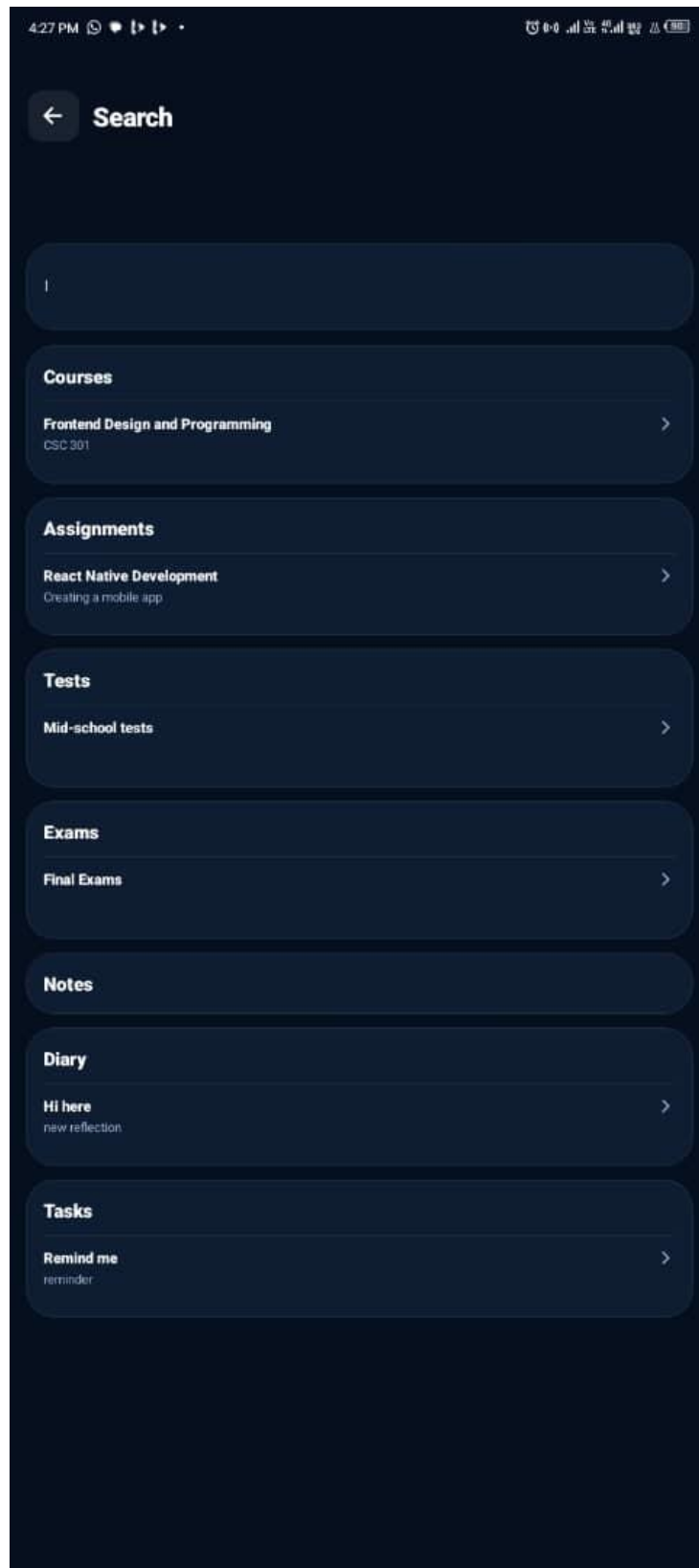
4.4.15 Search Module

As the volume of academic information increases throughout the semester, locating specific records manually becomes increasingly inefficient. The Search module addresses this challenge by providing centralized search functionality across multiple academic resources maintained within the application.

Students may search for courses, assignments, notes, diary entries, tasks, examinations, study sessions, and other supported records using keywords or partial text matches. The backend processes search requests efficiently before returning only records belonging to the authenticated user.

This capability improves application usability by reducing the time required to locate specific academic information.

Figure 4.15: Global Search Interface



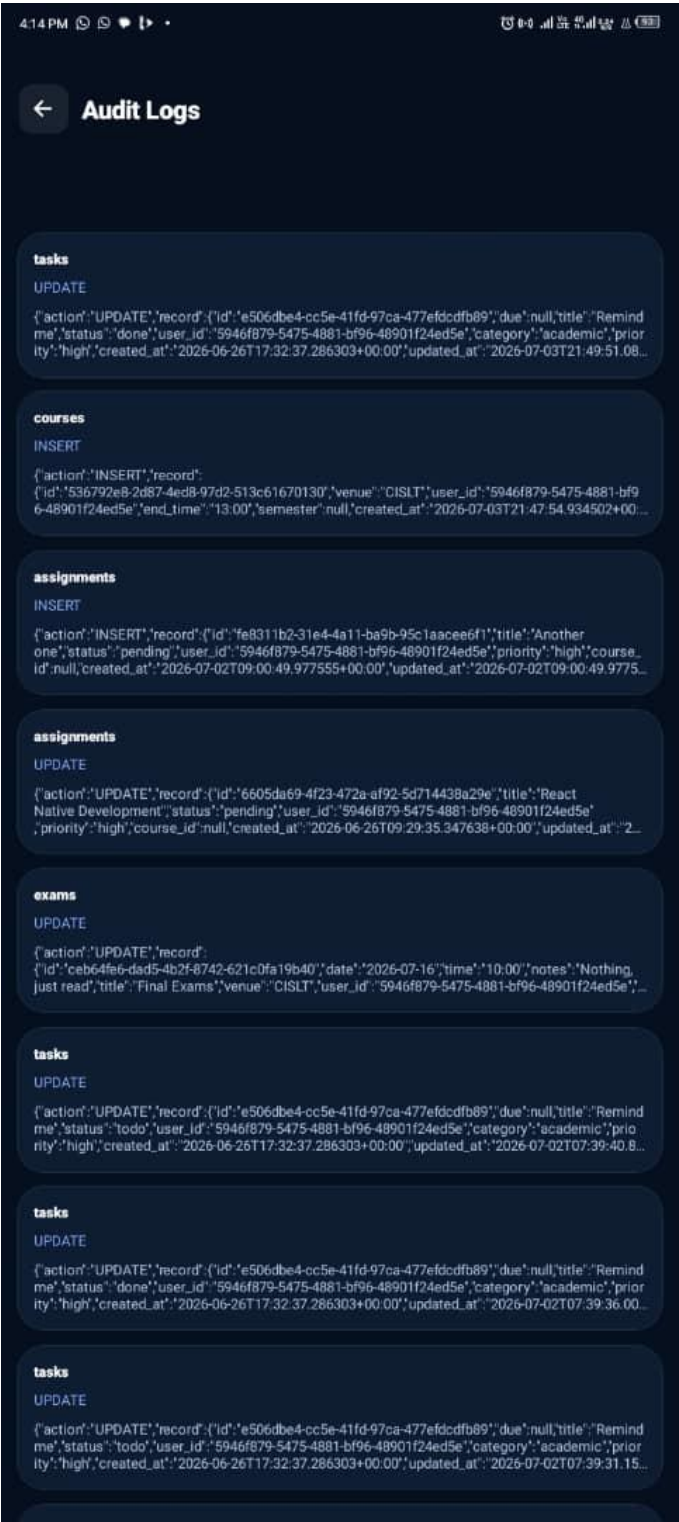
4.4.16 Analytics Module

The Analytics module provides students with summarized information regarding their academic activities and overall engagement within the application. Rather than presenting isolated records, the analytics subsystem aggregates information collected from various modules to generate meaningful statistics and performance indicators.

Examples of information presented by the analytics module include completed tasks, pending assignments, attendance summaries, study duration, upcoming assessments, grade distributions, and overall academic activity trends.

By presenting academic information in summarized graphical and numerical formats, the analytics module assists students in monitoring their academic progress and identifying areas requiring additional attention.

Figure 4.16: Academic Analytics Dashboard



4.4.17 Settings Module

Application settings constitute an important component of modern software systems because they provide users with the flexibility to personalize application behaviour according to their individual preferences. Rather than enforcing a fixed operational configuration for every user, settings modules enable applications to adapt to different usage patterns, accessibility requirements, notification preferences, and interface customizations. According to Nielsen (2020), user control and flexibility are fundamental usability principles that improve user satisfaction and overall system effectiveness.

The Settings module of the Academic Companion for Education (ACE) application was implemented to provide users with centralized control over various application preferences and account-related configurations. This module serves as the primary location through which students can customize their experience while ensuring that application behaviour aligns with their individual academic needs.

The module communicates securely with the backend services to retrieve and update user-specific configuration data stored within the Supabase database. Each authenticated user maintains an independent set of preferences, allowing personalized application behaviour across multiple devices while preserving data consistency.

Several configurable options are supported within the Settings module, including notification preferences, application behaviour, account management, privacy-related settings, and other configurable features implemented throughout the system. Changes made through the settings interface are validated before being transmitted to the backend, ensuring that invalid configuration values cannot be stored within the database.

The modular implementation of the Settings component also supports future extensibility. Additional configuration options may be introduced without requiring major modifications to existing application modules because settings are managed independently from the application's core business logic.

From a software engineering perspective, isolating application configuration within a dedicated module improves maintainability by separating user preferences from functional application

components. This architectural approach simplifies future feature expansion while reducing coupling between configuration management and operational modules.

Figure 4.17: Application Settings Interface

The screenshot displays a mobile application interface for editing a user profile. The main form, titled "Edit Profile", contains the following fields:

- FULL NAME:** A text input field containing "Prince Techture".
- INSTITUTION:** A dropdown menu showing "University of Ilorin".
- FACULTY:** A dropdown menu showing "Communication & Information Sciences".
- DEPARTMENT:** A dropdown menu showing "Information & Communication Science".
- LEVEL:** A section header for the next field.

A modal dialog titled "Select Department" is currently open, showing a list of available departments:

- Computer Science
- Information & Communication Science
- Library & Information Science
- Mass Communication
- Telecommunication Science

The modal includes a "Close" button in the top right corner. Below the modal, a "Cancel" button is visible.

4.4.18 User Profile Module

User profile management represents one of the foundational services implemented within the ACE application because every academic record maintained by the system is associated with an authenticated user account. Effective profile management allows students to maintain accurate personal information while supporting personalization, authentication, authorization, and data ownership throughout the application.

The User Profile module provides authenticated users with a centralized interface for viewing and updating personal information associated with their accounts. Information maintained within the profile includes personal identification details, account credentials, profile photographs where applicable, institutional information, and other user-specific data required by various application modules.

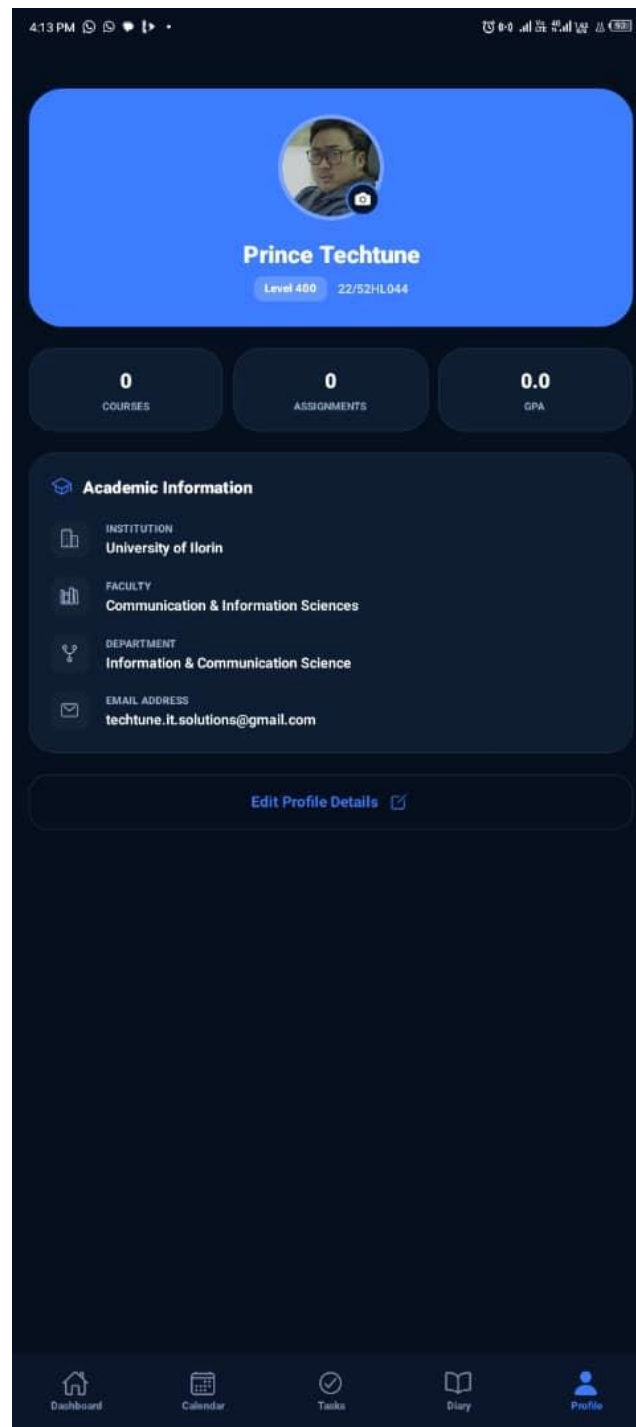
Whenever profile information is modified, update requests are transmitted securely to the backend through RESTful API endpoints implemented using the NestJS framework. Incoming requests undergo validation before being processed to ensure that submitted information satisfies predefined data integrity rules. Successfully validated updates are subsequently synchronized with the Supabase database, making the modified information immediately available throughout the application.

The profile module also serves as the identity provider for other functional modules within the ACE ecosystem. Academic records including assignments, attendance records, diary entries, study sessions, grades, notifications, notes, calendar events, and tasks are all associated with the authenticated user's unique identifier. This relational structure enables the backend to retrieve only records belonging to the currently authenticated user while maintaining data isolation and improving application security.

Integration with the authentication subsystem further ensures that profile information remains protected from unauthorized access. Authentication tokens generated during user login are verified before profile information can be viewed or modified, thereby enforcing access control policies implemented throughout the application.

The implementation of a dedicated profile management module contributes significantly to software usability by enabling students to maintain accurate personal information while supporting secure identity management across all application features.

Figure 4.18: User Profile Interface



4.5 System Deployment

System deployment is the process of transferring a completed software application from the development environment to an operational environment where it becomes accessible to its intended users. According to Sommerville (2021), deployment represents the final stage of software implementation, during which the developed system is installed, configured, tested in its production environment, and made available for operational use. A well-planned deployment strategy contributes significantly to software availability, maintainability, scalability, reliability, and ease of future updates.

Following the successful implementation and verification of the Academic Companion for Education (ACE) application, the various software components were deployed to cloud-based hosting platforms to ensure high availability, centralized management, and efficient delivery of services to users. Rather than deploying all components within a single environment, the application adopts a **distributed deployment architecture**, in which the mobile application, backend services, web landing page, database infrastructure, and notification services operate as independent components while communicating securely over the Internet using RESTful web services and HTTPS protocols.

The adoption of a distributed deployment model offers several advantages over traditional monolithic deployments. Each software component can be maintained, updated, scaled, or redeployed independently without interrupting the operation of the remaining system components. This modular deployment strategy also improves fault isolation, allowing failures within one service to have minimal impact on the availability of other application services. Furthermore, independent deployment enables future expansion of the system by allowing additional services to be introduced without requiring significant modifications to the existing deployment infrastructure.

The overall deployment architecture adopted for the ACE application is illustrated in Figure 4.19.

Figure 4.19: Deployment Architecture of the ACE Application

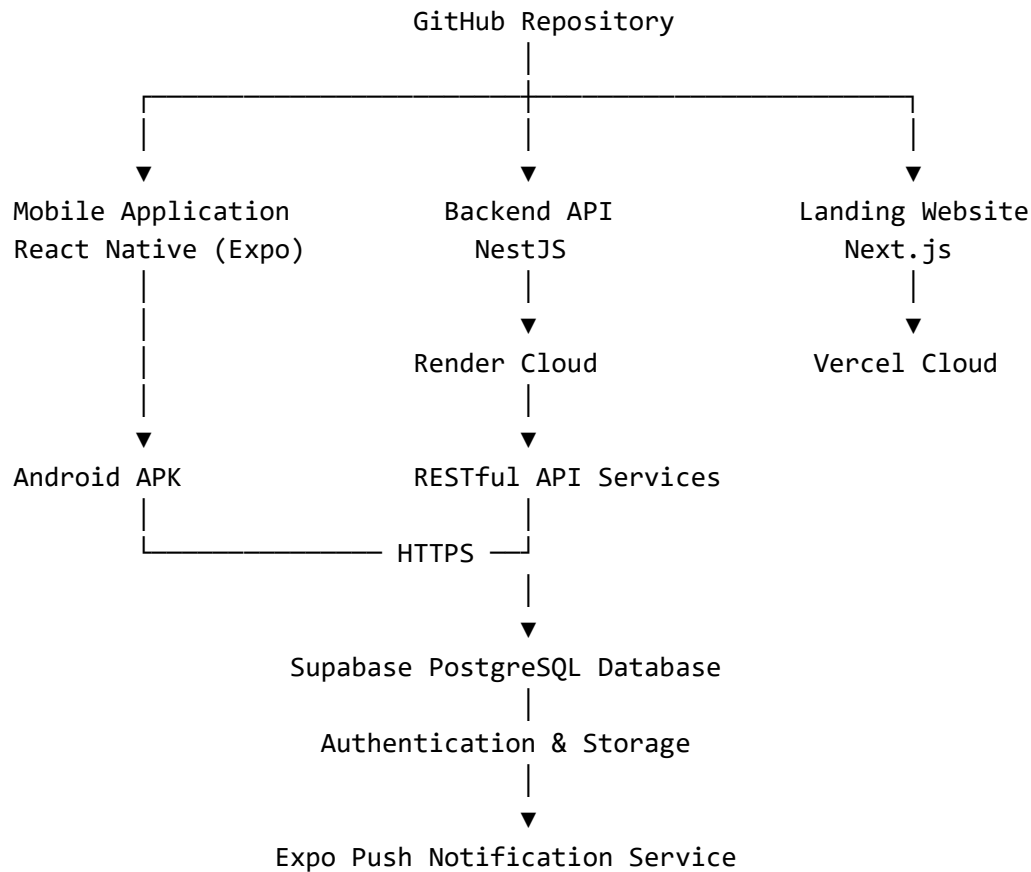


Figure 4.19: Deployment Architecture of the Academic Companion for Education (ACE) Application

As illustrated in Figure 4.19, the application's source code was maintained within a centralized GitHub repository using Git version control. The repository served as the primary source of truth for all software components, including the mobile application, backend services, web landing page, and technical documentation. Source code modifications were committed and synchronized through GitHub, enabling effective version management, collaboration, rollback of previous implementations where necessary, and automated deployment through integrated cloud services.

The backend application was deployed using **Render**, a cloud application hosting platform that supports Node.js applications and automated deployment directly from Git repositories. Render continuously monitors the backend source code repository and automatically rebuilds and redeploys the application whenever approved changes are pushed to the designated deployment

branch. This Continuous Integration and Continuous Deployment (CI/CD) workflow reduces manual deployment effort while ensuring that the production environment remains synchronized with the latest stable version of the application.

The public landing website was deployed using **Vercel**, a cloud platform optimized for modern frontend applications. Vercel provides automatic build pipelines, global content delivery, secure HTTPS hosting, and seamless integration with GitHub repositories. Through this deployment model, updates made to the website are automatically compiled and published after successful verification, allowing users to access the latest version of the website without manual intervention.

The mobile application was implemented using **React Native** through the **Expo Framework** and was tested extensively using **Expo Go** during the development phase. Expo Go enabled rapid iterative testing by allowing application updates to be reflected immediately on physical Android devices without requiring complete application recompilation. For production deployment, release builds of the application were generated using **Expo Application Services (EAS Build)**, which provides cloud-based compilation of React Native applications for Android and iOS platforms. EAS Build simplifies native application generation by managing the underlying Android build environment while producing optimized installation packages suitable for distribution.

Persistent application data was managed through **Supabase**, which provides a managed PostgreSQL relational database together with integrated authentication, storage services, row-level security policies, and secure API access. Hosting the database within a managed cloud infrastructure eliminates the administrative overhead associated with maintaining dedicated database servers while ensuring reliable data availability, automatic backups, scalability, and secure access to user information.

The application also integrates **Expo Notifications** to support scheduled and push notification services. Notification requests generated by the application are processed through Expo's notification infrastructure before being delivered to users' mobile devices. This service enables students to receive reminders relating to assignments, examinations, study sessions, attendance, and other academic activities even when the application is not actively running.

Conclusively, the deployment architecture adopted for the ACE application provides a secure, scalable, and maintainable cloud-based infrastructure capable of supporting future application growth. By separating the mobile application, backend services, web interface, notification system, and database into independent deployment units, the architecture promotes software reliability, simplifies maintenance, and facilitates continuous improvement throughout the application's lifecycle.

4.5.1 Development Workflow and Build Process

The development of the Academic Companion for Education (ACE) application followed a structured software engineering workflow that emphasized code quality, maintainability, reproducibility, and continuous validation throughout the implementation process. Rather than focusing solely on feature development, the implementation process incorporated project initialization, dependency management, static type checking, project validation, local testing, cloud-based application building, version control, and deployment. This workflow ensured that the application remained stable, scalable, and suitable for production deployment.

The development activities were performed primarily within **Visual Studio Code (VS Code)**, which served as the Integrated Development Environment (IDE) throughout the project. VS Code provided source code editing, integrated terminal support, debugging tools, Git integration, syntax highlighting, intelligent code completion, and extension support for TypeScript, React Native, NestJS, and other technologies used in the implementation. The integrated terminal was extensively utilized for executing project initialization commands, dependency installation, compilation, testing, build generation, and deployment operations.

Project Initialization

Development of the mobile application commenced with the creation of a new Expo project using the Expo Command Line Interface (CLI). Expo provides a standardized environment for developing React Native applications while reducing the complexity associated with native Android and iOS configuration (Expo, 2025).

The initial project structure was generated using the following command:

```
npx create-expo-app Ace
```

Executing this command automatically generated the project directory, configured the initial application structure, installed the default dependencies, enabled TypeScript support, and created the foundation upon which subsequent application features were implemented.

Similarly, the backend service was initialized using the NestJS Command Line Interface, which automatically generated the initial project architecture, application modules, configuration files, and development environment required for implementing the server-side application.

```
pnpm dlx @nestjs/cli new backend --package-manager pnpm
```

The generated backend project adopted the modular architecture provided by NestJS, enabling individual application features such as authentication, notifications, assignments, attendance, courses, and analytics to be developed independently while remaining integrated within a unified server application.

Dependency Installation and Package Management

Following project initialization, the required third-party libraries and development dependencies were installed using modern JavaScript package managers. Throughout the implementation process, **npm** and **pnpm** were utilized to install, update, and maintain project dependencies required by both the frontend and backend applications.

Common commands executed during dependency management included:

```
npm install  
pnpm install
```

Additional packages required for implementing specific features, such as navigation, notifications, authentication, animations, cloud communication, and user interface components, were installed as development progressed. Package management ensured that all required libraries were properly versioned and available throughout the software development lifecycle.

Local Development Environment

During implementation, the mobile application was executed locally using the Expo development server. Running the development server enabled rapid testing on both Android emulators and physical mobile devices through **Expo Go**, thereby significantly reducing development time by supporting hot reloading and immediate reflection of source code modifications.

The development server was started using the following command:

```
npx expo start
```

Upon execution, the Metro Bundler was launched, generating a Quick Response (QR) code that could be scanned using the Expo Go application installed on a physical Android device. This workflow allowed application changes to be viewed immediately without rebuilding the entire application after every modification.

The backend application was executed locally using the NestJS development server, which automatically recompiles and restarts the server whenever source code changes are detected.

```
npm run start:dev
```

This development workflow significantly accelerated feature implementation and debugging by providing immediate feedback during software development.

Static Type Checking and Project Validation

To maintain software reliability and reduce runtime errors, static analysis tools were executed regularly throughout development. Because both the frontend and backend applications were implemented using TypeScript, compile-time type checking was performed to verify type correctness before generating production builds.

The TypeScript compiler was executed using the following command:

```
npx tsc --noEmit
```

The `--noEmit` option instructs the TypeScript compiler to perform complete type analysis without generating JavaScript output files. This process detects missing properties, incompatible data types, undefined variables, incorrect function signatures, and other compile-time inconsistencies before application deployment.

In addition to static type checking, the overall health of the Expo project was verified using the Expo Doctor utility.

```
npx expo-doctor
```

Expo Doctor performs automated validation of the application environment by examining installed dependencies, SDK compatibility, package versions, native module configuration, and project consistency. Executing this command throughout development helped identify dependency conflicts, unsupported packages, and configuration issues that could potentially affect application stability or production builds.

Version Control and Source Code Management

Throughout implementation, **Git** served as the project's distributed version control system, while **GitHub** functioned as the centralized repository for maintaining source code, documentation, configuration files, and project history. Source code modifications were committed regularly using descriptive commit messages that documented newly implemented features, bug fixes, performance improvements, and configuration changes.

Version control provided several advantages during development, including source code backup, historical change tracking, collaborative development, rollback capability, and seamless integration with automated deployment platforms such as Render and Vercel. Furthermore, technical documentation maintained using Markdown files was version-controlled alongside the application source code, ensuring consistency between implementation and documentation.

Cloud-Based Application Build Process

Following successful implementation and testing, production-ready Android application packages were generated using **Expo Application Services (EAS Build)**. Unlike traditional local Android builds, EAS Build performs native compilation within Expo's managed cloud infrastructure, eliminating the need for extensive local Android build configuration while ensuring consistent build environments across development machines.

Before generating application builds, the project was authenticated with the Expo platform and configured for cloud builds using the following commands:

```
eas login  
eas build:configure
```

Preview builds intended for testing purposes were generated using:

```
eas build --platform android --profile preview
```

Upon successful verification of the preview release, production-ready Android application packages were generated using:

```
eas build --platform android --profile production
```

The generated application package was subsequently installed on physical Android devices for final verification before deployment. Utilizing the cloud-based EAS Build service simplified native application compilation while ensuring reproducible builds suitable for production distribution.

The structured development workflow adopted throughout the implementation of the ACE application reflects contemporary software engineering practices by integrating project initialization, dependency management, continuous validation, static type checking, version control, automated cloud builds, and systematic testing into a unified development process. This disciplined workflow contributed significantly to the reliability, maintainability, and overall quality of the completed software system.

4.6 System Testing

Software testing is a critical phase of the Software Development Life Cycle (SDLC) that involves evaluating software to determine whether it satisfies its specified functional and non-functional requirements. The primary objective of testing is to identify defects, verify correctness, evaluate reliability, and ensure that the completed software performs according to user expectations before deployment. According to Pressman and Maxim (2020), software testing provides confidence that the implemented system satisfies both its design specifications and operational requirements while reducing the likelihood of failures during practical use.

For the Academic Companion for Education (ACE) application, testing was not treated as a separate activity performed only after implementation had been completed. Instead, testing was integrated into every stage of development using an iterative development approach. Each newly implemented feature was verified immediately after development before being integrated with the remaining components of the application. This continuous testing strategy made it possible to identify implementation errors early, simplify debugging, improve software quality, and reduce the cost associated with correcting defects at later stages of development.

Testing activities covered both the frontend and backend components of the application. The mobile application was evaluated using physical Android devices together with Android emulators, while backend services and the web landing page were tested within local development environments before deployment to their respective cloud hosting platforms. After deployment, additional verification was performed to ensure that all distributed components communicated correctly over the Internet.

Several categories of software testing were conducted throughout the implementation process.

4.6.1 Unit Testing

Unit testing focused on verifying the correctness of individual software components independently before they were integrated into the complete application. Each module developed within the NestJS backend was implemented and verified separately to ensure that its business logic, validation rules, database operations, and API responses functioned correctly in isolation.

Similarly, reusable frontend components such as forms, navigation screens, validation utilities, state management logic, and helper functions were individually tested during implementation to verify correct rendering, input validation, and expected behaviour.

Whenever modifications were introduced into existing modules, the affected components were re-tested before integration to ensure that newly introduced features did not adversely affect previously implemented functionality. This incremental testing approach significantly reduced regression errors during development.

4.6.2 Integration Testing

Following successful verification of individual modules, integration testing was conducted to evaluate communication between the major software components comprising the ACE application.

Since the application follows a client-server architecture, integration testing primarily focused on verifying the successful exchange of information between the mobile application, backend services, cloud database, authentication services, and notification infrastructure.

The following software integrations were successfully verified during implementation:

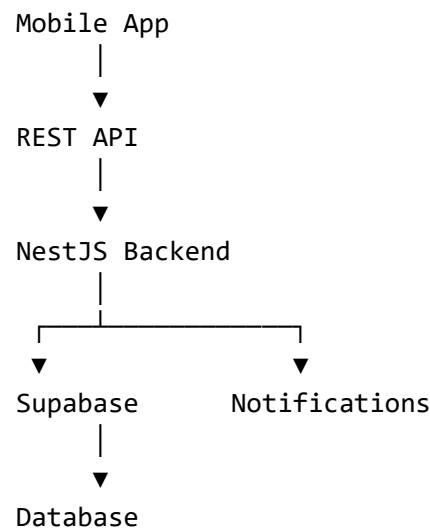
- Mobile Application ↔ NestJS Backend API
- Backend API ↔ Supabase PostgreSQL Database
- Backend API ↔ Supabase Authentication
- Mobile Application ↔ Expo Notification Service
- Backend API ↔ Analytics Module
- Backend API ↔ Artificial Intelligence Module
- Landing Website ↔ Backend API (where applicable)

Authentication requests, database transactions, notification scheduling, profile synchronization, CRUD operations, AI-assisted requests, analytics generation, and academic record management

were all tested repeatedly to verify that information flowed correctly between interconnected software components.

Figure 4.20 illustrates the communication architecture verified during integration testing.

Figure 4.20 System Integration Flow



Successful integration testing confirmed that information transmitted from the mobile application was correctly processed by the backend services, securely stored within the Supabase PostgreSQL database, and subsequently returned to users whenever requested. Furthermore, notification requests generated by the application were successfully transmitted to the Expo Notification Service for delivery to users' mobile devices.

4.6.3 Functional Testing

Functional testing was conducted to determine whether every implemented feature performed according to the functional requirements identified during system analysis. Unlike unit testing, which evaluates individual software components independently, functional testing examines the observable behaviour of the completed application from the perspective of an end user.

Each major module within the ACE application was tested by performing realistic academic tasks that students are expected to execute during normal use. These included user registration, authentication, course creation, assignment management, attendance recording, note creation,

diary management, examination scheduling, study session planning, AI interactions, analytics generation, notification scheduling, and academic search operations.

Testing confirmed that each implemented feature produced the expected results without introducing functional inconsistencies or unexpected system failures.

Table 4.4 summarizes the outcome of the functional testing performed during implementation.

Table 4.4: Summary of the Functional Testing Performed on the Application

Test Case	Expected Result	Outcome
User Registration	Account created successfully	Passed
User Login	Successful authentication	Passed
Add Course	Course stored successfully	Passed
Add Assignment	Assignment created	Passed
Attendance Recording	Attendance saved	Passed
Notes Creation	Note saved	Passed
Diary Entry	Entry stored	Passed
Create Study Session	Session scheduled	Passed
Add Examination	Examination created	Passed
Add Task	Task created	Passed
Notification Scheduling	Notification delivered	Passed
Search Records	Matching records returned	Passed
AI Assistant	Responses generated	Passed
Analytics Dashboard	Statistics displayed	Passed

After successful execution of each test case, the corresponding database records were inspected through the Supabase dashboard to verify that information was correctly stored, updated, retrieved, and deleted where applicable. Similarly, API responses were verified using Postman before frontend integration to ensure consistency between backend responses and user interface behaviour.

4.6.4 Compatibility Testing

Compatibility testing was performed to determine whether the application maintained consistent functionality across different hardware devices, software environments, operating systems, and execution platforms.

During development, the mobile application was tested extensively using **Expo Go** on both physical Android hardware and Android emulators generated through **Android Studio**. Physical device testing was conducted using an **Infinix Smart 7 HD** smartphone, enabling evaluation under realistic operating conditions including mobile network connectivity, notification delivery, application responsiveness, touchscreen interactions, and hardware performance.

In addition to physical device testing, Android Studio emulators were used to evaluate application behaviour across different virtual device configurations and Android API levels. Emulator testing simplified debugging activities while allowing rapid verification of interface layouts, navigation behaviour, and screen responsiveness without requiring repeated installation on physical hardware.

The web landing page was tested locally during development using the integrated development server before deployment. Cross-browser compatibility testing was subsequently performed using **Google Chrome** and **Microsoft Edge** to verify responsive layouts, navigation, content presentation, hyperlink functionality, and overall user experience.

Similarly, backend services were executed and verified within the local development environment prior to deployment on Render. RESTful API endpoints were tested using Postman and browser-based requests to confirm successful request processing, authentication, database communication, and response generation before production deployment.

The results of compatibility testing demonstrated that the application maintained consistent functionality across all evaluated platforms and development environments.

4.6.5 Performance Testing

Performance testing was conducted to evaluate the responsiveness, stability, and operational efficiency of the ACE application under normal usage conditions.

Performance evaluation focused primarily on application startup time, API response latency, database query execution, authentication speed, notification scheduling, and synchronization of academic records between the frontend and backend.

Throughout testing, the mobile application demonstrated satisfactory responsiveness during navigation between application screens, retrieval of academic records, creation of new data entries, and synchronization with cloud services. Backend API requests processed through the NestJS server returned responses within acceptable timeframes under normal testing conditions, while PostgreSQL database operations performed through Supabase exhibited reliable query execution with no significant delays.

Notification scheduling and delivery using Expo Notifications also performed reliably during testing, with scheduled reminders being successfully delivered to the test device according to their configured trigger times.

Overall, performance testing indicated that the developed application provides satisfactory responsiveness and operational stability suitable for its intended academic environment.

4.6.6 Usability Testing

Usability testing was conducted to evaluate the ease with which prospective users could understand, navigate, and interact with the application without requiring extensive technical knowledge or formal training.

Prior to software implementation, the application's user interface was designed and prototyped using **Figma**, allowing interface layouts, navigation flows, typography, colour selection, spacing, and user interactions to be evaluated before coding commenced. The availability of interactive prototypes facilitated iterative design improvements and reduced interface-related issues during software implementation.

Following implementation, practical usability testing was performed by interacting with the completed application under realistic usage conditions. Typical academic tasks including account registration, authentication, course creation, attendance recording, assignment scheduling, note management, diary creation, AI interactions, academic search, and reminder configuration were executed repeatedly to assess navigation efficiency and overall usability.

The results indicated that users were able to complete common academic activities with minimal learning effort. Navigation between application modules remained intuitive, interface components were consistently organized, and visual feedback provided adequate confirmation following user interactions.

The combination of iterative Figma prototyping, continuous usability evaluation during development, and practical testing on physical devices contributed significantly to improving the overall user experience of the completed application.

4.7 System Evaluation

The completed Academic Companion for Education (ACE) application was evaluated against the objectives established in Chapter One as well as the overall aim of this research, which was to design and implement a mobile application for academic scheduling and reminders for University of Ilorin students.

The evaluation indicates that the developed system successfully satisfies this objective by providing a centralized mobile platform through which students can organize lecture schedules, manage assignments, create academic reminders, record attendance, plan study sessions, manage examinations, maintain academic notes, monitor grades, and receive timely notifications regarding upcoming academic activities.

Furthermore, the incorporation of University of Ilorin academic information during user registration enables the application to provide a more institution-oriented experience by allowing students to organize their academic activities within the context of their faculty, department, level of study, and programme of study.

The modular software architecture, cloud-based deployment model, secure authentication mechanisms, responsive user interface, and centralized academic management capabilities collectively demonstrate that the developed application satisfies both the functional and non-functional requirements identified during system analysis while supporting the academic scheduling and reminder needs of University of Ilorin students.

The modular software architecture, cloud-based deployment model, secure authentication mechanisms, responsive user interface, and centralized academic management capabilities

collectively demonstrate that the implementation fulfills both the functional and non-functional requirements identified during system analysis.

4.8 Chapter Summary

This chapter has presented the implementation, deployment, testing, and evaluation of the Academic Companion for Education (ACE) application. It described the software development environment, architectural design, implementation of the major functional modules, cloud deployment strategy, software testing procedures, and system evaluation conducted throughout the project.

The implementation demonstrates that the integration of React Native, Expo, NestJS, Supabase, PostgreSQL, Render, Vercel, and modern software engineering practices resulted in a scalable, secure, maintainable, and user-friendly academic companion application capable of supporting students in organizing and managing their academic activities effectively.

REFERENCES

- Allen, D. (2015). *Getting Things Done: The Art of Stress-Free Productivity*. Penguin Books.
- Ecma International. (2017). *ECMA-404: The JSON Data Interchange Syntax* (2nd ed.). Retrieved June 30, 2026, from <https://ecma-international.org/publications-and-standards/standards/ecma-404/>
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems* (7th ed.). Pearson Education.
- Expo. (2025). *Create a project*. <https://docs.expo.dev/get-started/create-a-project/>
- Expo. (2025). *EAS Build Documentation*. <https://docs.expo.dev/build/introduction/>
- Expo. (2025). *Expo Application Services (EAS Build) Documentation*. <https://docs.expo.dev/build/introduction/>
- Expo. (2025). *Expo CLI Documentation*. <https://docs.expo.dev/more/expo-cli/>
- Expo. (2025). *Expo Doctor*. <https://docs.expo.dev/develop/tools/>
- Expo. (2025). *Expo Documentation*. Expo Developers. Retrieved June 30, 2026, from <https://docs.expo.dev/>
- Figma Inc. (2025). *Figma Documentation*. Retrieved June 30, 2026, from <https://help.figma.com/>
- Git Contributors. (2025). *Git Documentation*. Retrieved June 30, 2026, from <https://git-scm.com/doc>
- GitHub, Inc. (2025). *GitHub Documentation*. Retrieved June 30, 2026, from <https://docs.github.com/>
- Meta Platforms, Inc. (2025). *React Native Documentation*. React Native Team. Retrieved June 30, 2026, from <https://reactnative.dev/docs/getting-started>
- Microsoft. (2025). *TypeScript Documentation*. Microsoft Corporation. Retrieved June 30, 2026, from <https://www.typescriptlang.org/docs/>
- Mozilla Foundation. (2025). *MDN Web Docs*. Retrieved June 30, 2026, from <https://developer.mozilla.org/>
- NestJS. (2025). *NestJS CLI Documentation*. <https://docs.nestjs.com/cli/overview>
- NestJS. (2025). *NestJS Documentation*. NestJS. Retrieved June 30, 2026, from <https://docs.nestjs.com/>

Node.js Contributors. (2025). *Node.js Documentation*. OpenJS Foundation. Retrieved June 30, 2026, from <https://nodejs.org/docs/latest/api/>

PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. Retrieved June 30, 2026, from <https://www.postgresql.org/docs/>

Postman, Inc. (2025). *Postman Learning Center*. Retrieved June 30, 2026, from <https://learning.postman.com/>

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Render Services, Inc. (2025). *Render Documentation*. Retrieved June 30, 2026, from <https://render.com/docs>

Sommerville, I. (2021). *Software Engineering* (11th ed.). Pearson Education.

Supabase Inc. (2025). *Supabase Documentation*. Retrieved June 30, 2026, from <https://supabase.com/docs>

Vercel Inc. (2025). *Vercel Documentation*. Retrieved June 30, 2026, from <https://vercel.com/docs>

WHATWG. (2025). *HTML Living Standard*. Retrieved June 30, 2026, from <https://html.spec.whatwg.org/>

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter presents the summary, findings, evaluation, and concluding aspects of the design and implementation project titled "Design and Implementation of a Mobile Application for Academic Scheduling and Reminders for University of Ilorin Students." It provides an overview of the problem addressed by the project, the software solution designed and implemented to solve the identified challenges, the extent to which the project objectives were achieved, the major challenges encountered during system development, the solutions adopted throughout implementation, the limitations of the completed system, and recommendations for future improvements.

The chapter also evaluates the contribution of the developed Academic Companion for Education (ACE) mobile application as a practical software solution for improving academic scheduling, reminder management, and overall academic organization among University of Ilorin students. Finally, it concludes by highlighting the significance of the project to students, educational institutions, and future developments in the application of mobile technologies to academic management.

5.2 Summary of the Study

The increasing academic demands placed on university students have made effective organization, scheduling, and time management essential for achieving academic success. Students are often required to manage lecture schedules, coursework, assignments, examinations, attendance records, study plans, project deadlines, and personal academic notes simultaneously. In many cases, these activities are managed using paper-based records or multiple unrelated digital applications, resulting in fragmented information management, missed deadlines, poor organization, and reduced academic productivity.

This design and implementation project, titled "Design and Implementation of a Mobile Application for Academic Scheduling and Reminders for University of Ilorin Students," was undertaken to address these challenges through the development of a centralized mobile application capable of assisting students in organizing and managing their academic activities more effectively. The developed solution, named the Academic Companion for Education (ACE), serves

as an integrated academic management platform that combines scheduling, reminders, academic record management, and productivity tools within a single mobile application.

To align the application with its intended users, the system was customized to reflect the academic structure of the University of Ilorin. During user registration and profile configuration, students select institution-specific information such as their faculty, department, level of study, and course of study from records seeded into the application's database. These academic details enable the application to organize student records within an institutional context while providing a foundation for future integration with university-specific academic services and administrative systems.

The design and implementation of the application followed established software engineering principles, including modular software architecture, cloud-based infrastructure, iterative software development, centralized data management, and continuous testing throughout the Software Development Life Cycle (SDLC). The user interfaces were first designed and prototyped using Figma before implementation commenced. The mobile application was then developed using React Native with the Expo Framework, enabling cross-platform compatibility from a single codebase. Backend services were implemented using the NestJS framework, while Supabase provided PostgreSQL database services, authentication, storage, and security policies. Additional technologies including TypeScript, Git, GitHub, Render, Vercel, Expo Application Services (EAS), and Expo Notifications collectively supported the successful design, implementation, testing, and deployment of the system.

The completed ACE application incorporates numerous functional modules that directly support the objectives of this project. These include secure user authentication, academic profile management, course management, lecture and academic scheduling, assignment tracking, attendance recording, examination scheduling, study session planning, task management, diary management, digital note-taking, grade management, intelligent search, academic analytics, notification services, profile management, settings management, and an integrated Artificial Intelligence (AI) assistant for academic support. Collectively, these modules provide students with a centralized environment for managing virtually every aspect of their academic activities.

Comprehensive implementation, testing, deployment, and system evaluation confirmed that the developed application satisfies both its functional and non-functional requirements. The integration of cloud-based technologies enables secure authentication, centralized data synchronization, reliable reminder delivery, efficient backend communication, and a scalable architecture capable of supporting future expansion. Furthermore, the modular implementation adopted throughout the project allows additional features and institutional services to be incorporated with minimal modifications to the existing system.

Overall, the successful design and implementation of the Academic Companion for Education (ACE) demonstrate that modern mobile technologies and cloud computing platforms can be effectively utilized to solve the challenges associated with academic scheduling and reminder management among university students. The completed application fulfills the primary aim of this project by providing University of Ilorin students with a centralized, secure, scalable, and user-friendly mobile application that supports effective academic planning, improves time management, enhances productivity, and assists students in keeping track of their academic responsibilities through intelligent scheduling and reminder services.

5.3 Achievement of the Research Objectives

The primary objective of this study was to design and implement a mobile application capable of assisting university students in organizing and managing their academic activities within a unified digital environment. Based on the implementation, testing, and evaluation presented in the preceding chapters, the objectives of the study were successfully achieved.

The first objective was to investigate the challenges associated with existing approaches to academic management among students. The findings presented in Chapter Two established that many students experience difficulties arising from fragmented academic information, inconsistent scheduling methods, missed assignment deadlines, poor study planning, and limited access to integrated academic management tools. These findings provided the justification for the development of the ACE application.

The second objective was to design an intuitive, responsive, and user-friendly academic companion capable of supporting essential academic activities. This objective was achieved through the use of Figma during the interface design stage, enabling iterative prototyping and

refinement before software implementation commenced. The resulting interface emphasizes simplicity, consistency, accessibility, and ease of navigation.

The third objective was to implement a functional cross-platform mobile application capable of integrating multiple academic management services within a single software platform. This objective was achieved through the successful implementation of the ACE application using React Native, Expo, NestJS, TypeScript, Supabase, and PostgreSQL. The completed system supports authentication, course management, attendance tracking, assignments, notes, diary entries, study sessions, examinations, grades, notifications, analytics, search functionality, artificial intelligence assistance, and cloud synchronization.

The fourth objective was to evaluate the developed system through systematic testing and validation procedures. Unit testing, integration testing, functional testing, compatibility testing, performance evaluation, and usability assessment confirmed that the implemented system satisfies both its functional and non-functional requirements. Testing demonstrated reliable communication between the mobile application, backend services, database infrastructure, notification services, and supporting cloud platforms.

The successful implementation of these objectives demonstrates that the ACE application provides a practical, reliable, and scalable solution for supporting students in the effective management of their academic activities.

5.4 Challenges Encountered During Development

The development of the Academic Companion for Education (ACE) application involved several technical, operational, and infrastructural challenges that are commonly encountered during the implementation of modern cloud-based mobile applications. Although contemporary development frameworks and cloud platforms provide numerous tools for accelerating software development, they also introduce constraints relating to service limitations, deployment restrictions, cloud resource management, platform compatibility, and operational costs.

Throughout the implementation of the project, these challenges required careful analysis, continuous experimentation, and the adoption of alternative engineering solutions to ensure the successful completion of the system. The major challenges encountered during the development of the ACE application are discussed below.

5.4.1 Cloud Service Limitations

One of the most significant challenges encountered during the implementation of the application involved the limitations imposed by free-tier cloud services. Since the project was developed as an academic project without dedicated financial sponsorship, several cloud platforms were initially utilized under their free usage plans.

Although these services provided sufficient resources during the early stages of development, various operational restrictions became evident as the application evolved into a feature-rich academic management platform.

For example, cloud platforms generally impose limits on storage capacity, network bandwidth, processing resources, concurrent connections, monthly build quotas, and application execution time. As the number of implemented features increased, these restrictions occasionally affected development efficiency, testing activities, and deployment workflows.

These limitations highlighted the practical trade-offs between free development resources and production-ready cloud infrastructure, emphasizing the importance of proper infrastructure planning during software engineering projects.

5.4.2 Backend Hosting Constraints

The backend services for the ACE application were deployed using the free tier of Render.

Although Render provides an excellent environment for hosting Node.js applications with automated deployment, the free service plan automatically suspends inactive applications after a period of inactivity to conserve computing resources.

Consequently, the first API request made after prolonged inactivity often experiences additional latency while the server instance is restarted. This behaviour, commonly referred to as **cold start latency** or **server sleeping**, temporarily increases response time for the initial request before normal performance resumes.

During testing, this occasionally resulted in delayed authentication requests and slower loading of application data immediately after prolonged inactivity. Although this behaviour did not affect the correctness of application functionality, it slightly influenced the perceived responsiveness experienced by users during initial application startup.

Commercial hosting plans eliminate this limitation by maintaining continuously active server instances; however, these plans incur recurring operational costs that were beyond the scope of the present project.

5.4.3 Mobile Application Build Restrictions

Another significant challenge involved the generation of production-ready Android application packages.

The application utilized Expo Application Services (EAS Build) to compile native Android builds through Expo's cloud infrastructure. While EAS significantly simplifies mobile application compilation by eliminating the need for local Android build environments, the free development plan imposes limitations on the number of cloud builds available within a given period.

Because each production build consumes one build allocation, repeatedly generating new application packages after every minor software modification proved inefficient during development.

Furthermore, cloud-based compilation introduces queue times that vary depending on server availability and the number of concurrent builds being processed by the platform.

As development progressed, build queues occasionally increased the waiting time before newly generated application packages became available for testing.

5.4.4 Application Update Management

Closely related to build limitations was the challenge of efficiently distributing software updates during development and testing.

Generating a completely new Android installation package after every modification would have significantly slowed the implementation process while consuming the available monthly cloud build allocation.

To address this challenge, the project adopted **Over-the-Air (OTA) updates** using Expo Updates for modifications that affected JavaScript, TypeScript, styling, user interface behaviour, and application logic.

OTA updates allowed compatible application changes to be delivered directly to installed devices without rebuilding the native Android application package. Native builds were therefore reserved

primarily for updates involving native dependencies, SDK upgrades, or platform-specific configuration changes.

This approach significantly accelerated feature testing while improving development efficiency and reducing unnecessary cloud build consumption.

5.4.5 Third-Party Service Dependency

The architecture of the ACE application integrates multiple cloud services, including Supabase, Render, Vercel, Expo Application Services, Expo Notifications, and GitHub.

While this distributed architecture improves scalability and maintainability, it also introduces dependencies on the availability and operational status of external cloud providers.

Temporary service outages, scheduled maintenance periods, network interruptions, or changes to platform policies could potentially affect one or more application services despite the application's internal software functioning correctly.

This dependency illustrates one of the practical considerations associated with cloud-native software development.

5.4.6 Database Resource Constraints

Supabase was selected as the cloud database platform because of its integrated PostgreSQL services, authentication system, storage capabilities, and developer-friendly APIs.

However, the free service tier includes limitations relating to database storage capacity, file storage quotas, bandwidth allocation, and computational resources.

Although these limitations were sufficient throughout the implementation and testing phases of this study, large-scale institutional deployment involving thousands of concurrent users would require migration to higher-capacity commercial service plans capable of supporting increased workloads.

5.4.7 Deployment Cost Considerations

Although numerous modern cloud platforms provide generous free development plans, deploying production software capable of supporting large numbers of users inevitably incurs operational costs.

Maintaining the ACE application in a production environment requires expenditure relating to backend hosting, managed databases, cloud storage, domain registration, mobile application distribution, notification services, monitoring, analytics, and future infrastructure scaling.

Consequently, long-term sustainability of the application would require an appropriate funding model or institutional support to ensure continued operation and maintenance.

Table 5.1: Estimated Production Infrastructure Cost

Service	Development Tier	Production Tier	Approximate Monthly Cost
GitHub	Free	Free/Team	\$0–\$4 per user
Render	Free	Starter	~\$7
Supabase	Free	Pro	~\$25
Vercel	Hobby	Pro	~\$20
Expo EAS Build	Free	Production	Based on usage
Domain Name	Annual	Annual	~\$10–20/year

5.5 Solutions Adopted

To ensure the successful completion of the project despite the challenges encountered, several practical software engineering solutions were implemented throughout development.

Rather than allowing infrastructure limitations to delay implementation, alternative approaches were adopted to optimize resource utilization while maintaining software quality.

5.5.1 Adoption of Modular Software Architecture

The application was implemented using a modular architecture provided by the NestJS framework together with organized React Native project structures.

This modular organization simplified debugging, reduced software complexity, encouraged code reuse, and allowed individual application modules to be developed and tested independently.

The resulting architecture also supports future expansion with minimal disruption to existing application functionality.

5.5.2 Continuous Version Control

Git and GitHub were employed throughout development to maintain complete source code history.

Frequent commits ensured that implementation progress could be recovered whenever unexpected software issues occurred.

Version control also simplified collaboration, documentation, deployment automation, and source code maintenance.

5.5.3 Incremental Testing

Rather than postponing software testing until implementation had been completed, continuous testing was performed after the implementation of each functional module.

This incremental testing strategy significantly reduced debugging effort by allowing implementation errors to be identified immediately after introduction.

5.5.4 Adoption of OTA Updates

To reduce dependency on frequent cloud builds, Over-the-Air (OTA) updates were adopted for software modifications that did not require changes to native application code.

This significantly accelerated development by allowing application improvements to be distributed almost immediately without generating entirely new Android application packages.

Native production builds were reserved for updates requiring native dependencies or platform configuration changes.

5.5.5 Efficient Cloud Resource Utilization

Cloud resources were carefully managed throughout development to remain within the operational limits of the available free service plans.

Unused resources were removed where appropriate, unnecessary build generation was avoided, database queries were optimized, and application assets were managed efficiently to minimize bandwidth consumption.

5.5.6 Documentation Throughout Development

Comprehensive technical documentation was maintained throughout the software development lifecycle using Markdown documentation stored within the project's GitHub repository.

This documentation included installation procedures, project architecture, API descriptions, development guidelines, deployment instructions, implementation notes, and project configuration information.

Maintaining synchronized documentation alongside the source code significantly improved maintainability while facilitating future software enhancement and knowledge transfer.

5.6 Limitations of the Developed System

Although the ACE application successfully satisfies the objectives of this study, certain limitations remain.

The current implementation primarily targets Android devices, with iOS support requiring additional platform-specific testing and deployment. Integration with institutional information systems such as Learning Management Systems (LMS), university portals, and examination management platforms has not yet been implemented. AI-assisted features depend on external services and internet connectivity, limiting their functionality in offline environments. In addition, cloud-based free-tier services introduce constraints on scalability, availability, storage capacity, and build frequency that would need to be addressed before large-scale deployment.

Operational costs also represent a practical limitation. While the project was implemented largely using free development tiers, sustained production use would require paid subscriptions for backend hosting, database scaling, build services, monitoring, custom domains, and potentially application distribution. These recurring expenses should be considered when planning institutional or commercial deployment.

5.7 Conclusion

This research successfully achieved its primary objective of designing and implementing a mobile application for academic scheduling and reminders for University of Ilorin students. Through a systematic software engineering approach encompassing requirements analysis, system design, implementation, testing, deployment, and evaluation, the Academic Companion for Education (ACE) application was developed as a comprehensive academic management platform capable of supporting students throughout their academic journey.

Unlike conventional reminder or note-taking applications that address isolated aspects of academic organization, the developed system integrates multiple academic management services within a unified mobile environment. These services include user authentication, academic scheduling, course management, assignment tracking, attendance recording, study session planning, examination scheduling, grade management, diary entries, digital note-taking, intelligent search,

notification services, academic analytics, and artificial intelligence-assisted academic support. By consolidating these functionalities into a single application, the system minimizes information fragmentation and enables students to manage their academic responsibilities more efficiently.

To further align the application with the objectives of this study, institutional information relating to the University of Ilorin, including faculties, departments, levels of study, and academic programmes, was incorporated into the user registration and profile configuration process. This institutional customization enables students to organize their academic activities within the context of their university environment while providing a foundation for future integration with university-specific academic systems and services.

The implementation further demonstrates the effectiveness of contemporary software engineering technologies in solving real-world educational problems. The integration of React Native, Expo, NestJS, TypeScript, Supabase, PostgreSQL, Figma, GitHub, Render, Vercel, and other supporting technologies resulted in a scalable, maintainable, secure, and user-friendly software solution developed in accordance with established software engineering principles and industry practices.

Although certain limitations associated with cloud service subscriptions, hosting resources, build quotas, and infrastructure costs remain, these constraints do not diminish the practical value of the developed application. Instead, they provide clear opportunities for future enhancement as the application evolves beyond its current implementation. Future work may include deeper integration with University of Ilorin academic systems, lecturer and departmental portals, institutional timetables, enhanced artificial intelligence capabilities, offline synchronization, cross-platform optimization, and broader deployment across other tertiary institutions.

In conclusion, the Academic Companion for Education (ACE) application demonstrates that mobile technologies can be effectively leveraged to improve academic scheduling, reminder management, and educational organization among university students. The successful design and implementation of the system therefore validates the feasibility of developing a centralized academic companion capable of supporting the daily academic activities of University of Ilorin students while providing a scalable foundation for future institutional and commercial expansion.